Feeding charts and schema mutations

Somnio x Google A2UI finance showcase — generated from the Markdown in docs/guides/, which is the source of truth.

finance-google-a2ui · index.html

Feeding charts — how a client or the agent gets chart data

Audience: anyone writing a client for this backend (Flutter, a web prototype, a notebook) or wiring an agent to it. **Source of truth:** ADR-K (server-side projections), ADR-E (the display/raw pair), `.claude/rules/backend/presentation.md` and `.claude/rules/frontend/contract-binding.md`. The companion guide, `schema-mutations.md`, explains how the shapes described here are allowed to change.

Every response quoted below was captured from a running backend on 2026-09-22 (`GET /health` → `{"status":"ok","version":"local+b7e78bf"}`) with the commands shown; the screenshots are the `frontend/` client of the same build, signed in as the demo user. Long decimals and the bearer token are elided as `…`, nothing else is edited.

1. The one idea

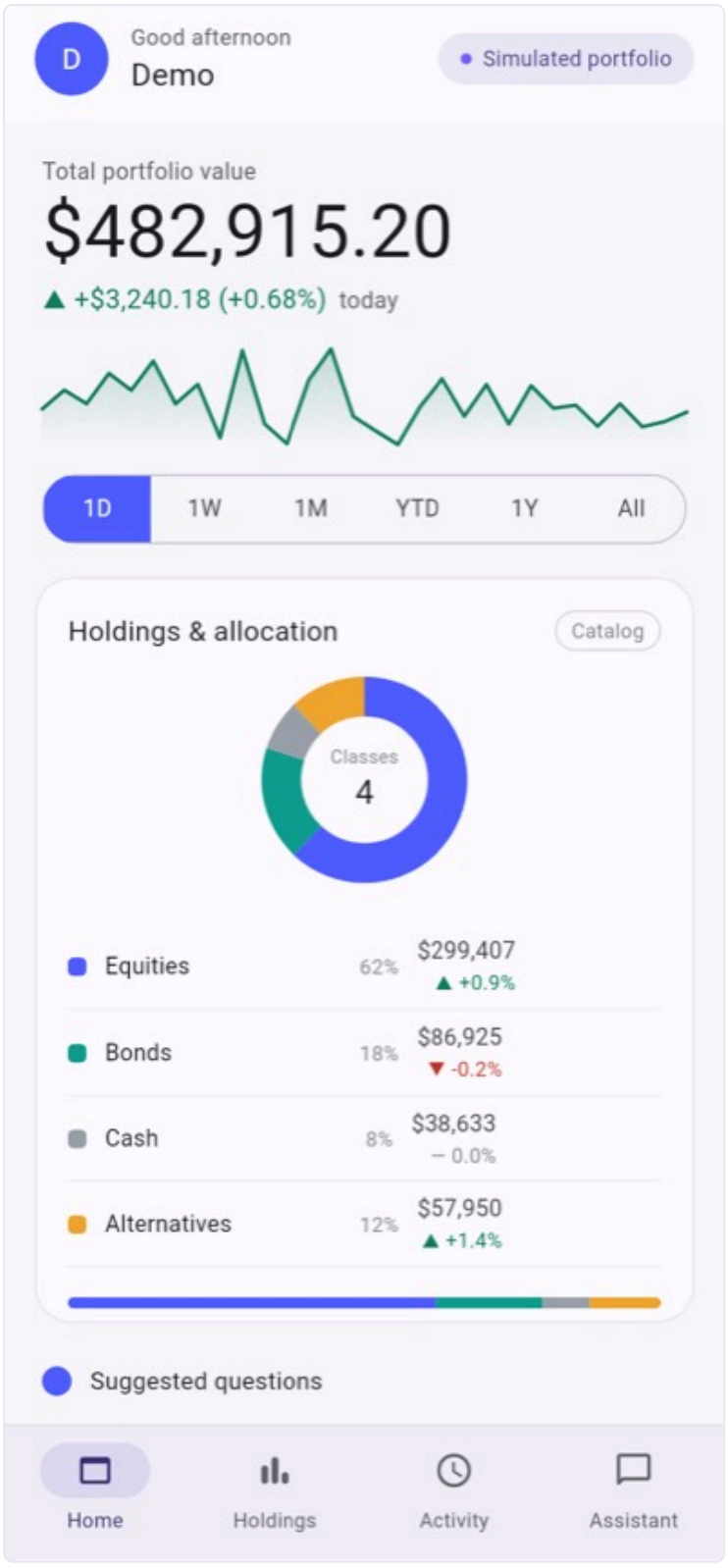
The backend shapes every payload; the client binds it. A chart is not "the entity plus a mapping in Dart" — it is a **projection**: a named, versioned, pure function *(portfolio, options) → payload*, registered once under an id such as `attribution.waterfall.v1`, and served with **every figure already formatted**. The client puts strings on screen and draws geometry from the raw numbers beside them. It never rounds, sums, sorts or formats anything, and neither does the model.

The eleven projections this build serves:

View id	Component the client registers	What it draws
`portfolio.summary.v1`	`PortfolioSummary`	The Home hero: total, change over a range, 30-day sparkline, curated questions, top movers
`holdings.allocation.v1`	`HoldingsSummary`	Donut by holding, asset class or sector
`holdings.positions.v1`	`PositionList`	The Holdings tab, one row per position
`activity.timeline.v1`	`ActivityTimeline`	The Activity tab, grouped and filterable
`attribution.waterfall.v1`	`AttributionWaterfall`	Unrealised P/L walked from cost basis to market value

View id	Component the client registers	What it draws
`attribution.driver.v1`	`DriverDetail`	One holding's part in today's move (what a tapped bar opens)
`taxlots.table.v1`	`TaxLotTable`	The lots a proposed sale consumes
`scenario.comparison.v1`	`ScenarioComparison`	The same sale under FIFO / LIFO / HIFO
`payoff.curve.v1`	`RiskPayoffChart`	Linear spot payoff across a price range
`order.ticket.v1`	`OrderTicket`	Single-asset proposed order with editable inputs
`order.ticket.v2`	`MultiLegOrderTicket`	Multi-leg draft with stepper values and an impact panel

All eleven are served by the same two endpoints (`GET /v1/portfolios/{portfolio_id}/views` and `POST /v1/view-renders`), and all eleven appear in the catalog (`GET /v1/views`) — the list above was read from it, not typed.



The Home tab binds portfolio.summary.v1 and holdings.allocation.v1 (group_by=asset_class, headline=false)

Home on a 390 px viewport: `portfolio.summary.v1` *is the hero and sparkline,* `holdings.allocation.v1` *with* `group_by=asset_class` *is the donut. Every figure on screen is a* `display` *string from those two payloads.*

2. Authenticate

Every route except `/health`, `/readyz`, the OpenAPI document and the agent card needs `Authorization: Bearer <token>`. Locally, the preview sign-in mints one for an allow-listed e-mail:

```
TOKEN=$(curl -s -X POST localhost:8000/v1/preview-sign-ins \
  -H "Content-Type: application/json" -d '{"email":"demo@somnio.dev"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
```

```
HTTP/1.1 201 Created
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.…","token_type":"Bearer","expires_in":28800}
```

Without it, a view read answers an RFC 9457 problem document — the error shape every `/v1/**` route uses:

```
{"type":"https://errors.finance-a2ui/auth/missing-token","title":"Unauthorized","status":401,
 "detail":"No bearer token was
supplied.","instance":"/v1/portfolios/current/views","request_id":"1af2250f-…"}
```

Branch on the `type` suffix (`auth/missing-token`), never on `detail`, which is prose.

3. Discover what you can ask for: `GET /v1/views`

The catalog is the introspection surface (ADR-K D-8). One entry per registered projection, all of it derived from the registry — nothing here is hand-written, so it cannot disagree with what the read endpoint serves.

```
curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/v1/views \
  | python3 -c 'import sys,json; [print(v["id"], "→", v["component"]) for v in json.load(sys.stdin)]'
```

Each entry (`ViewDescriptor`) carries:

Key	What it is for
`id`	The versioned view id you send back as `view=`
`component`	The single string the Flutter client registers a widget under — read from the projection's own A2UI adapter
`summary`	One sentence of purpose
`endpoints`	Where it is served: both endpoints, for all eleven today
`options_schema`	The JSON Schema of the declared options — the **only** knobs you may send (§6)
`fields`	The frozen field inventory: one row per payload path with `type`, `presence`, `binding`, `state`, `input` (§5 and the companion guide)
`example`	A rendered example payload over the sample portfolio — what a component test binds
`prompt`	The lines the system prompt teaches the model about this view (spec 0034)

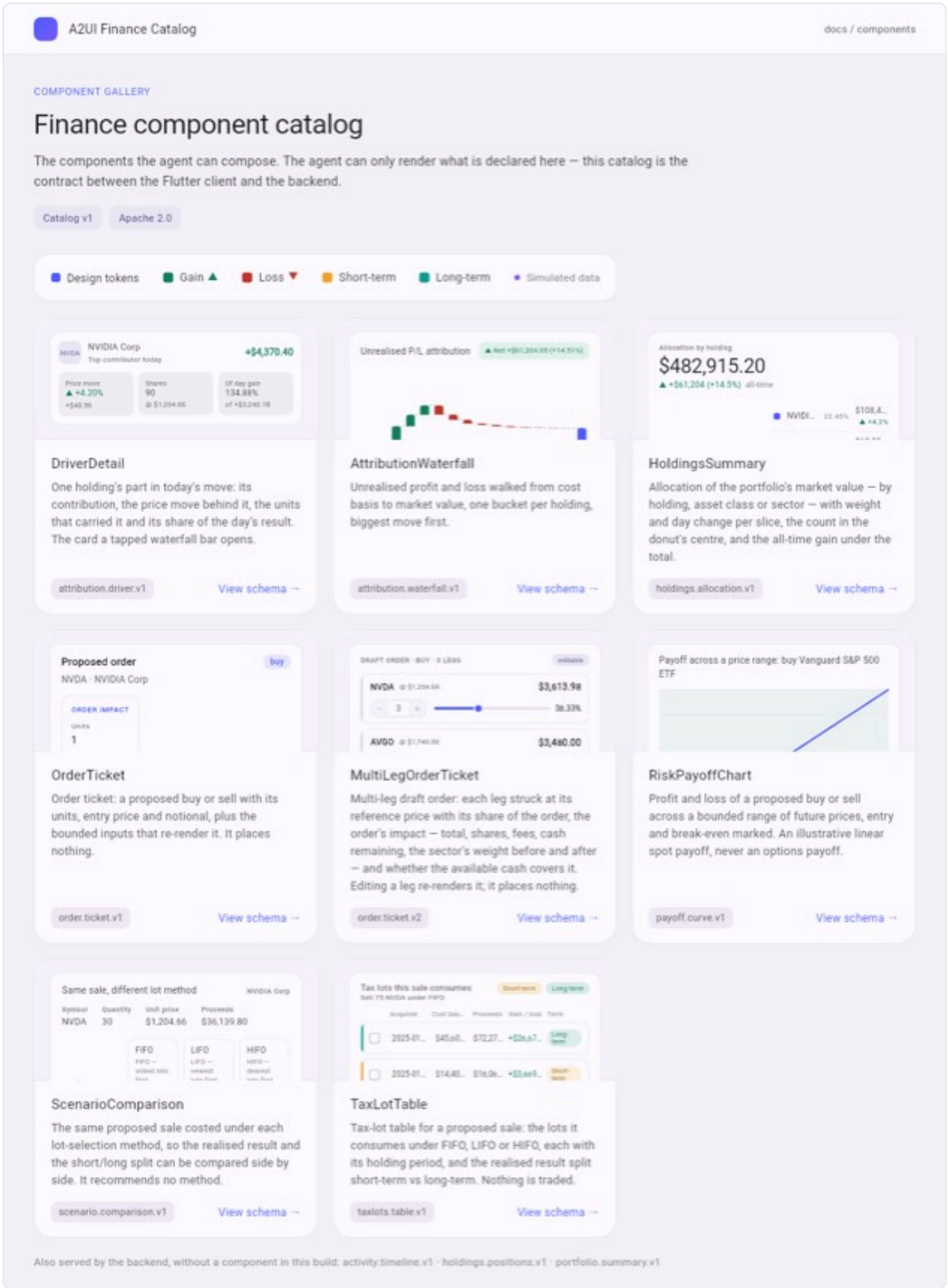
Key	What it is for
`guidance`	The designer's `use_when` / `avoid_when` pair, or `null` where design has not written one
`actions`	What a surface may send *back* as a new agent turn — declared choices, not free text (§8)

A slice of the waterfall's entry, as served:


```
{
  "id": "attribution.waterfall.v1",
  "component": "AttributionWaterfall",
  "summary": "Unrealised profit and loss walked from cost basis to market value, one bucket per
holding, biggest move first.",
  "endpoints": ["GET /v1/portfolios/{portfolio_id}/views", "POST /v1/view-renders"],
  "options_schema": {"properties": {"fields": {…}, "locale": {"enum": ["en-US", "es-AR", "de-DE"], …},
                    "precision": {"minimum": 0, "maximum": 8, …}, "currency": {"const": "USD", …}},
                    "additionalProperties": false, …},
  "fields": [
    {"path": "view",                "type": "const 'attribution.waterfall.v1'", "presence": "default
`\"attribution.waterfall.v1\"`", "binding": "constant",  "state": null,      "input": null},
    {"path": "title",               "type": "string ≤120 chars", "presence": "required",
"binding": "text",      "state": null,      "input": null},
    {"path": "net.direction",       "type": "enum up / down / flat", "presence": "required",
"binding": "enum",      "state": "variant", "input": null},
    {"path": "buckets",             "type": "array",             "presence": "default `[]`",
"binding": "container", "state": "empty",   "input": null},
    {"path": "buckets[].key",       "type": "string ≤32 chars",  "presence": "required",
"binding": "text",      "state": null,      "input": null},
    {"path": "buckets[].delta.amount","type": "decimal",          "presence": "required",
"binding": "raw",       "state": null,      "input": null},
    {"path": "buckets[].delta.display","type": "string ≤32 chars", "presence": "required",
"binding": "display",   "state": null,      "input": null},
    …
  ],
  "actions": [{"action": "drill.attribution.v1", "label": "Break this down", "arg": "driver",
"control": "choice",
              "choices": ["AAPL", "AGG", "AMD", "AVGO", "BND", "GLD", "JNJ", "MSFT", "NVDA", "PG",
"PYPL", "TIP", "TSLA", "USD", "VNQ", "VOO", "XOM"], "pattern": null}],
  "guidance": {"use_when": "the user asks WHY a value changed and the answer decomposes into signed
contributions…",
              "avoid_when": "a static breakdown of what is held now — `holdings.allocation.v1`
answers that. Never write the figures yourself…"},
  "prompt": "- attribution.waterfall.v1 — Unrealised profit and loss walked …\n  options: fields (list
of up to 8 (text), or omit it), locale (one of 'en-US', 'es-AR', 'de-DE'), precision (integer 0..8),
currency (only 'USD')\n  use when: …\n  do not use for: …",
```

```
  "example": {"view": "attribution.waterfall.v1", "title": "Unrealised P/L attribution", …}
}
```

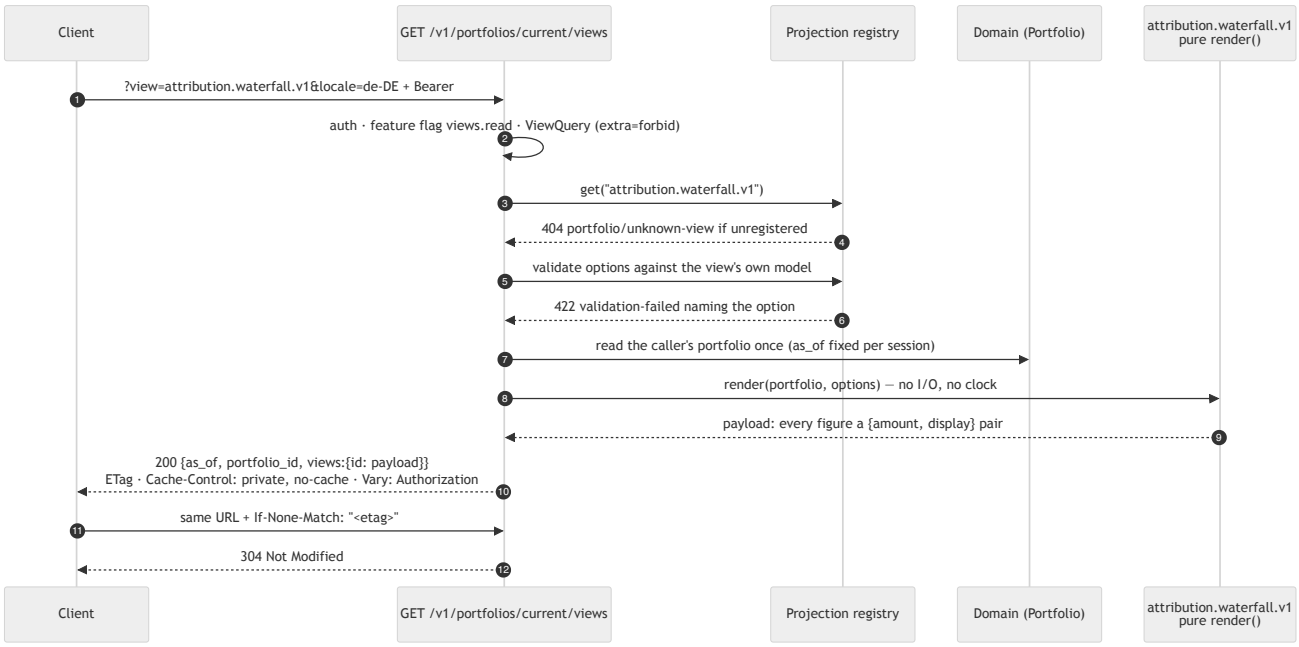
The same catalog is published twice more, from the same render pass, as checked-in files: `docs/catalog/CONTRACT.md` for the designer and `docs/catalog/view-catalog.snapshot.json` for the client's tests. The running app draws it as a gallery at `/#/catalog`, every card rendered from the entry's real `example`:



The component catalog page, rendered by the Flutter client from GET /v1/views

4. Read one chart: `GET /v1/portfolios/current/views?view=<id>`

`current` is the only portfolio id today; it resolves to the caller's own portfolio. The request path for one chart:



The call, and the envelope it answers (the `views` object is keyed by view id — the same shape whether you asked for one view or four):

```
curl -s -D - -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=attribution.waterfall.v1"
```

```
HTTP/1.1 200 OK
content-type: application/json
etag: "cc66d1e0eeb006c5d02329a0da79e304b8314d30e931f4a6f8630a81ba899ae5"
cache-control: private, no-cache
vary: Authorization
x-request-id: ea0d2447-0d0a-40cc-909b-a21d53c1e2ed

{
  "as_of": "2026-09-22T13:48:16.650046Z",
  "portfolio_id": "current",
  "views": {
    "attribution.waterfall.v1": {
      "view": "attribution.waterfall.v1",
      "title": "Unrealised P/L attribution",
      "start": {"label": "Cost basis",   "amount": {"amount": "421711.15…", "display":
"$421,711.15"}},
      "end":   {"label": "Market value", "amount": {"amount": "482915.20…", "display":
"$482,915.20"}},
      "net":   {"label": "Net",
```

```
                "amount":  {"amount": "61204.05…", "display": "+$61,204.05"},
                "percent": {"value": "0.14513263403161144778837362 97", "display": "+14.51%"},
                "direction": "up"},
      "buckets": [
        {"label": "NVIDIA Corp", "key": "NVDA",
         "delta":      {"amount": "34019.40…",  "display": "$34,019.40"},
         "cumulative": {"amount": "455730.55…", "display": "$455,730.55"},
         "direction": "up",
         "move":  {"value": "0.45725", "display": "+45.72%"},
         "units": {"units": "90.000000000000000000", "display": "90"},
         "price": {"amount": "1204.66", "display": "$1,204.66"},
         "detail": "+45.72% · 90 sh · $1,204.66",
         "selected_label": "NVDA selected"},
        {"label": "Apple Inc.", "key": "AAPL", "delta": {"amount": "29334.00…", "display":
"$29,334.00"}, …},
        … 12 more, largest move first, ending with {"key": "USD", "delta": {"display": "$0.00"}, …}
      ],
      "annotations": [
        {"label": "NVIDIA Corp", "text": "$34,019.40"},
        {"label": "Apple Inc.",  "text": "$29,334.00"},
        {"label": "Net",         "text": "$61,204.05"}
      ],
      "actions": [{"action": "drill.attribution.v1", "label": "Break this down", "arg": "driver",
                   "control": "choice", "choices": ["AAPL", "AGG", …, "XOM"], "pattern": null}]
    }
  }
}
```


Three things to notice, because they are the whole contract:

- `as_of` **travels in the envelope, as a string.** The client never reads its own clock for it (N-13).
- **The buckets arrive in the order the widget draws them.** `sort` is an option where a view offers one; the client never re-ranks rows (N-10).
- **The** `key` **is what a tap sends back.** `buckets[].key` (`"NVDA"`) is the value `drill.attribution.v1` accepts as its `driver`; the `label` is prose and cannot be sent (§8).

4.1 Revalidate: ETag and 304

The body is a pure function of *(subject, greeting, portfolio, views, options,* `as_of`*)*, so it carries a **strong** `ETag`. Echo it back verbatim — the quotes are part of the tag — and an unchanged body is not sent again:

```
curl -s -i -H "Authorization: Bearer $TOKEN" \
  -H 'If-None-Match: "cc66d1e0eeb006c5d02329a0da79e304b8314d30e931f4a6f8630a81ba899ae5"' \
  "localhost:8000/v1/portfolios/current/views?
view=holdings.allocation.v1&view=attribution.waterfall.v1"
```

```
HTTP/1.1 304 Not Modified
etag: "cc66d1e0eeb006c5d02329a0da79e304b8314d30e931f4a6f8630a81ba899ae5"
cache-control: private, no-cache
vary: Authorization
```

A 304 is a **success**: the client resolves it to the cached envelope. `Cache-Control` is `private, no-cache` on purpose — the body is subject-scoped, and a feature flag can switch a route off at any moment, so the client may keep the body but must ask before reusing it.

4.2 Several charts in one round trip

Repeat `view=` up to **4** times. One authentication, one session, one portfolio read, *N* pure renders, one keyed object (ADR-K D-17):

```
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?
view=holdings.allocation.v1&view=attribution.waterfall.v1" \
  | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d["as_of"], list(d["views"]))'
# 2026-09-22T13:48:16.650046Z ['holdings.allocation.v1', 'attribution.waterfall.v1']
```

The option set is **flat and shared**: `?view=a&view=b&precision=0` applies `precision` to both, and an option only one of them declares fails the whole request with a 422 (§6). The `frontend/` client mirrors the cap as `ViewsRepository.maxViewsPerRequest = 4` and chunks a screen that wants more.

4.3 Let the backend choose

Omit `view=` and the documented **selection rule** (ADR-K D-13) picks from the data: every projection scores its own `fitness` over the portfolio, the highest wins, and the endpoint's default (`holdings.allocation.v1`) is used only when nothing scores above zero. Against the seeded portfolio, which holds both winners and losers, the waterfall wins:

```
curl -s -H "Authorization: Bearer $TOKEN" "localhost:8000/v1/portfolios/current/views" \
  | python3 -c 'import sys,json; print(list(json.load(sys.stdin)["views"]))'
# ['attribution.waterfall.v1']
```

Eight of the eleven views have `fitness = 0`: they are asked for by id and never volunteered (a tax-lot table with no sale in mind would be noise).

4.4 Heterogeneous batches: `POST /v1/view-renders`

When the views span different portfolios or need **different options per view**, the batch is a resource (ADR-K D-18): up to **8** entries, a `201` whenever the batch itself was well formed, and **a status per entry**, so one bad view never sinks the others. Nothing durable is created, so there is no `Location` header.

```
cat > batch.json <<'EOF'
{"renders":[
```

```
  {"view":"portfolio.summary.v1",   "portfolio_id":"current", "options":{}},
  {"view":"payoff.curve.v1",        "portfolio_id":"current", "options":{"size":"2"}},
  {"view":"totally.invented.v9",    "portfolio_id":"current", "options":{}},
  {"view":"holdings.allocation.v1", "portfolio_id":"current", "options":{"top_n":99}}
]}
EOF
curl -s -i -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d @batch.json localhost:8000/v1/view-renders
```

```
HTTP/1.1 201 Created

{"results": [
  {"status": 200, "view": "portfolio.summary.v1", "portfolio_id": "current", "as_of": "2026-09-
22T13:48:16.650046Z",
   "payload": {"view": "portfolio.summary.v1", "greeting": "Good afternoon", …}},
  {"status": 200, "view": "payoff.curve.v1", "portfolio_id": "current", "as_of": "2026-09-
22T13:48:16.650046Z",
   "payload": {"view": "payoff.curve.v1", "title": "Payoff across a price range: buy Vanguard S&P 500
ETF", …}},
  {"status": 404, "view": "totally.invented.v9", "portfolio_id": "current",
   "error": {"type": "https://errors.finance-a2ui/portfolio/unknown-view", "title": "Not Found",
"status": 404,
             "detail": "view 'totally.invented.v9' is not registered"}},
  {"status": 422, "view": "holdings.allocation.v1", "portfolio_id": "current",
   "error": {"type": "https://errors.finance-a2ui/validation-failed", "title": "Unprocessable Entity",
"status": 422,
             "detail": "view 'holdings.allocation.v1' rejected option(s) ['top_n']"}}
]}
```

(`top_n` is bounded `1..20`; `99` is out of range, so the entry is refused by the projection's own options model — exactly as the `?view=` path would refuse it.) A batch is not revalidated: `ETag` covers the two `GET` reads only.

5. The pair convention — raw for the plot, display for the text

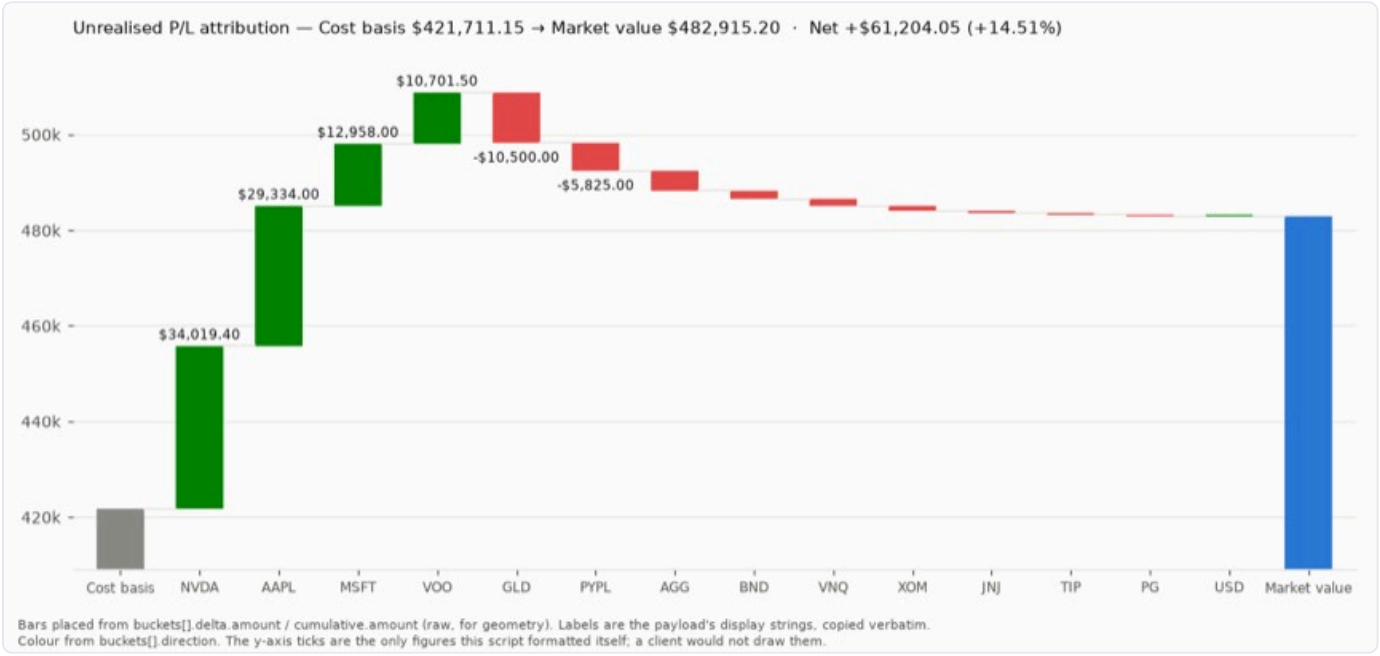
Every figure is an ADR-E **pair**: an exact decimal serialised as a string, beside the string the user reads.

```
"delta":   {"amount": "34019.40000000000000000000", "display": "$34,019.40"}
"percent": {"value":  "0.14513263403161144778837362 97", "display": "+14.51%"}
"units":   {"units":  "90.00000000000000000", "display": "90"}
```

Member	Binding in `fields[]`	The client may…	The client may never…
`display`	`display`	print it, verbatim, as the figure	prepend a sign or symbol, re-round, re-localise
`amount` / `value` / `units`	`raw`	turn it into a **coordinate** — a bar height, a point on a line, a donut angle	print it, sum it, compare it to pick a colour

The sign is **baked into** the display string (`"+$61,204.05"` , `"−$10,500.00"`). Colour comes from the supplied enum (`direction` , `trend` , `tone`), not from parsing the string. If a number must appear on screen and only a raw member carries it, **the payload is missing a field — raise it** (N-3); the `frontend/` client throws `PayloadContractException` naming the path rather than formatting the raw value.

Here is that rule made visible. Both charts below were drawn by a script from the payloads captured above: the marks are placed from the raw members, and **every label is a `display` string copied from the same payload**. The y tick labels on the waterfall are the one thing the script formatted itself, and they are the one thing a client would not draw.



attribution.waterfall.v1 drawn from the captured payload: bars from delta.amount and cumulative.amount, labels from display

![payoff.curve.v1 drawn from the captured payload: the line from points[].at.amount and value.amount, every tick a display string](assets/payoff_from_payload.png)

Why it matters for a chart specifically: a chart library that generates its own axis labels formats numbers inside a dependency, which breaks this rule silently. That is why `frontend/` ports d3's *arithmetic* (scales, ticks, layout) into a pure-Dart package and paints with `CustomPainter` , rather than adopting `fl_chart` or `graphic` (ADR-L D-13). The value-to-pixel mapping is the only arithmetic a client performs, and it is confined to one file (`raw_decimal.dart`) reachable only through `ViewPayload.geometry(path)` (ADR-L D-14).

6. Options: the only way a payload varies

Each view publishes a **bounded, defaulted, documented** options model (`options_schema` in the catalog). Send an option as a query parameter and the same view re-renders differently. Nothing else is interpreted — no field paths, no templates, no expressions (ADR-K D-3).

```
# German locale, whole dollars — every string re-renders, nothing else moves
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=attribution.waterfall.v1&locale=de-DE&precision=0"
# "start": {"label": "Cost basis", "amount": {"amount": "421711.15…", "display": "421.711 $"}}
# "net":   {…, "amount": {"display": "+61.204 $"}, "percent": {"display": "+15%"}, "direction": "up"}

# Trim the sections the widget does not bind (ADR-K D-21)
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=attribution.waterfall.v1&fields=buckets"
# "buckets": [… 14 …], "annotations": []

# Fewer slices, alphabetical, rest folded into Other
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=holdings.allocation.v1&top_n=3&sort=label"
# categories[].label → ["Apple Inc.", "Cash & sweep", "Exxon Mobil", "Other"]

# Slice the donut by sector instead of holding
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=holdings.allocation.v1&group_by=sector"
# "title": "Holdings & allocation", categories → Technology 75.12% · Financials 5.45% · Healthcare
1.55% · …
```

What a rejection looks like — the message names the **option**, never its value, so an echoed input cannot become a reflection vector:

```
# An option this view does not declare
curl -s -i -H "Authorization: Bearer $TOKEN" \
  "localhost:8000/v1/portfolios/current/views?view=attribution.waterfall.v1&top_n=3"
```

```
HTTP/1.1 422 Unprocessable Entity
{"type":"https://errors.finance-a2ui/validation-failed","title":"Unprocessable Entity","status":422,
 "detail":"view 'attribution.waterfall.v1' rejected option(s)
['top_n']","instance":"/v1/portfolios/current/views","request_id":"e7681db9-…"}
```

```
# A section this view does not offer
…?view=attribution.waterfall.v1&fields=nope
# 422 validation-failed · "unknown section(s) ['nope']; this view offers ['buckets', 'annotations']"
```

```
# Out of range, or a parameter the endpoint itself does not know (ViewQuery is extra="forbid")
…?view=attribution.waterfall.v1&precision=99
…?view=attribution.waterfall.v1&sparkle=1
# 422 malformed-request · "1 request field(s) failed validation"

# A view that does not exist
…?view=totally.invented.v9
# 404 portfolio/unknown-view · "view 'totally.invented.v9' is not registered"
```

Two rules for a client: send only ids from `GET /v1/views` (N-6) and only options the view's `options_schema` declares (N-7); do **not** re-validate a server bound with a stricter client rule beside it (N-8) — the bound the server publishes is the bound it enforces, and a test walks the registry to prove it.

7. Every projection, as served

One captured excerpt per view, trimmed to the shape. Field-by-field inventories live in `CONTRACT.md`.

`portfolio.summary.v1` — `PortfolioSummary`

Options: `locale`, `precision`, `currency`, `range`. The greeting and account are server-written; `spark[]` is thirty `{label, value}` observations.


```
{"view": "portfolio.summary.v1", "greeting": "Good afternoon", "account": {"name": "Demo", "initial":
"D"},
 "badge": "Simulated portfolio",
 "total":  {"label": "Total portfolio value", "amount": {"amount": "482915.20…", "display":
"$482,915.20"}},
 "change": {"amount": {"amount": "3240.18…", "display": "+$3,240.18"},
            "percent": {"value": "0.006754948381510465147841136276", "display": "+0.68%"}, "trend":
"up", "period": "today"},
 "spark": [{"label": "08/24/2026", "value": {"amount": "483925.09…", "display": "$483,925.09"}}, … 30
points …],
 "questions_title": "…", "questions": [...], "movers_title": "Top movers today", "movers": [...]}
```


`holdings.allocation.v1` — `HoldingsSummary`

Options: `top_n` (1..20), `include_other`, `sort`, `locale`, `precision`, `currency`, `group_by` (`holding | asset_class | sector`), `headline`. The Home card asks for `headline=false` because the shell already draws the hero.


```
{"view": "holdings.allocation.v1", "title": "Allocation by holding", "group_by": "holding",
 "total": {"label": "Total market value", "amount": {"amount": "482915.20…", "display":
"$482,915.20"}},
 "total_gain": {"amount": {"display": "+$61,204", …}, "percent": {"display": "+14.5%", …}, "trend":
"up", "period": "all-time"},
```

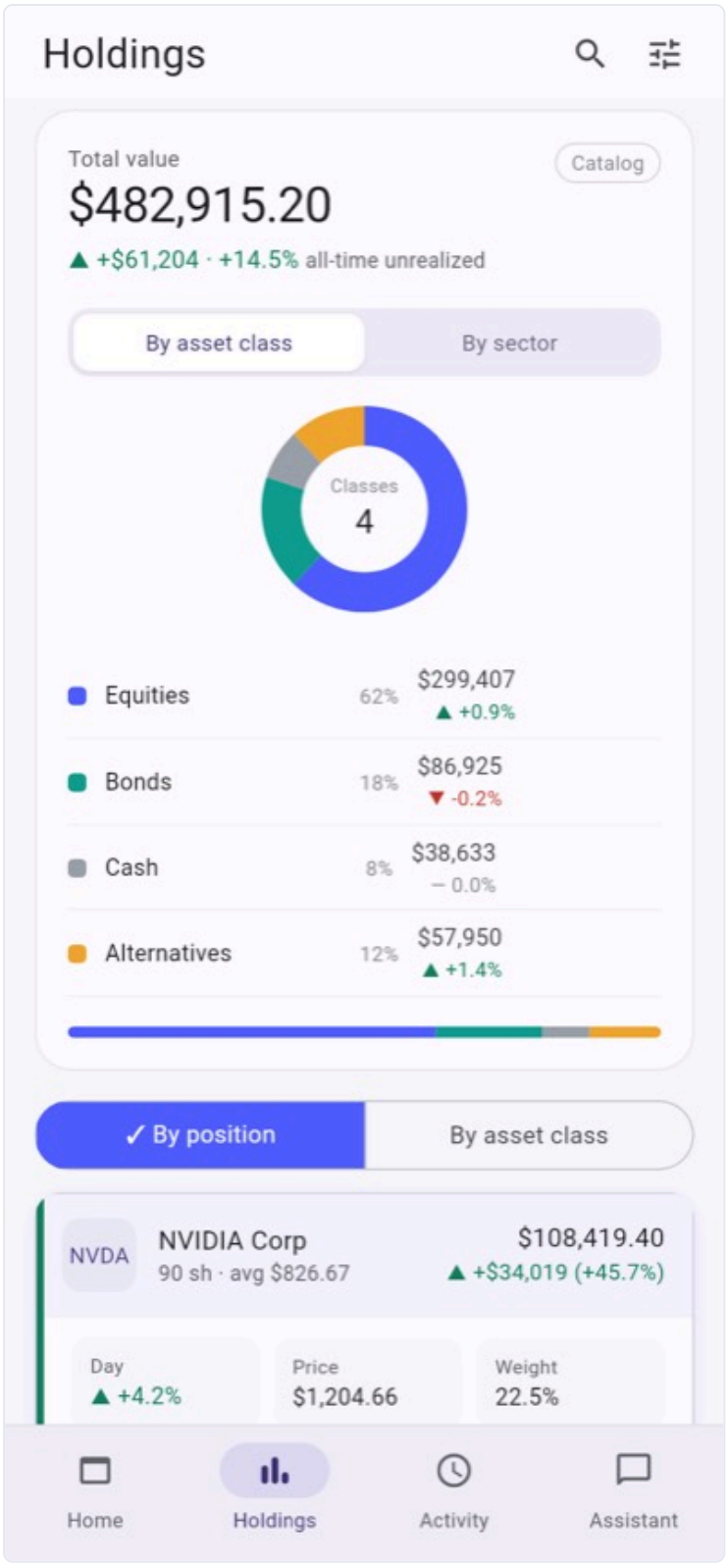
```
 "center": {"count": "14", "unit": "Holdings"},
 "categories": [
   {"label": "NVIDIA Corp", "key": "NVDA", "value": {"amount": "108419.40…", "display":
"$108,419.40"},
    "weight": {"value": "0.22451022456944821782375042 24", "display": "22.45%"},
    "change": {"value": "0.042003286912896808234581783 58", "display": "+4.2%"}, "trend": "up"},
   … {"label": "Other", "key": "other", "value": {"display": "$163,631.80", …}, "weight": {"display":
"33.88%", …}, …}]}
```


`center.count` is a **string** (`"14"`): a count the widget prints is text, not a number it may do arithmetic on.

`holdings.positions.v1` — `PositionList`

Options: `sort`, `locale`, `precision`, `currency`. Each row carries a server-written `detail` (`"90 sh · avg $826.67"`) and an `ask.text` the row's button sends to the assistant.


```
{"view": "holdings.positions.v1", "title": "Holdings",
 "total": {"label": "Total value", "amount": {"display": "$482,915.20", …}},
 "gain": {"amount": {"display": "+$61,204", …}, "percent": {"display": "+14.5%", …}, "trend": "up",
"period": "all-time unrealized"},
 "ask_label": "Ask the assistant about this",
 "positions": [{"symbol": "NVDA", "name": "NVIDIA Corp", "detail": "90 sh · avg $826.67",
                "value": {"display": "$108,419.40", …}, "unrealized": {"amount": {"display":
"+$34,019", …}, "percent": {"display": "+45.7%", …}, "trend": "up", "period": "all-time"},
                "day": {"value": "0.0420…", "display": "+4.2%"}, "day_trend": "up", "price":
{"display": "$1,204.66", …},
                "weight": {"display": "22.5%", …}, "is_cash": false, "ask": {"text": "Tell me about my
NVDA position"}}, …]}
```

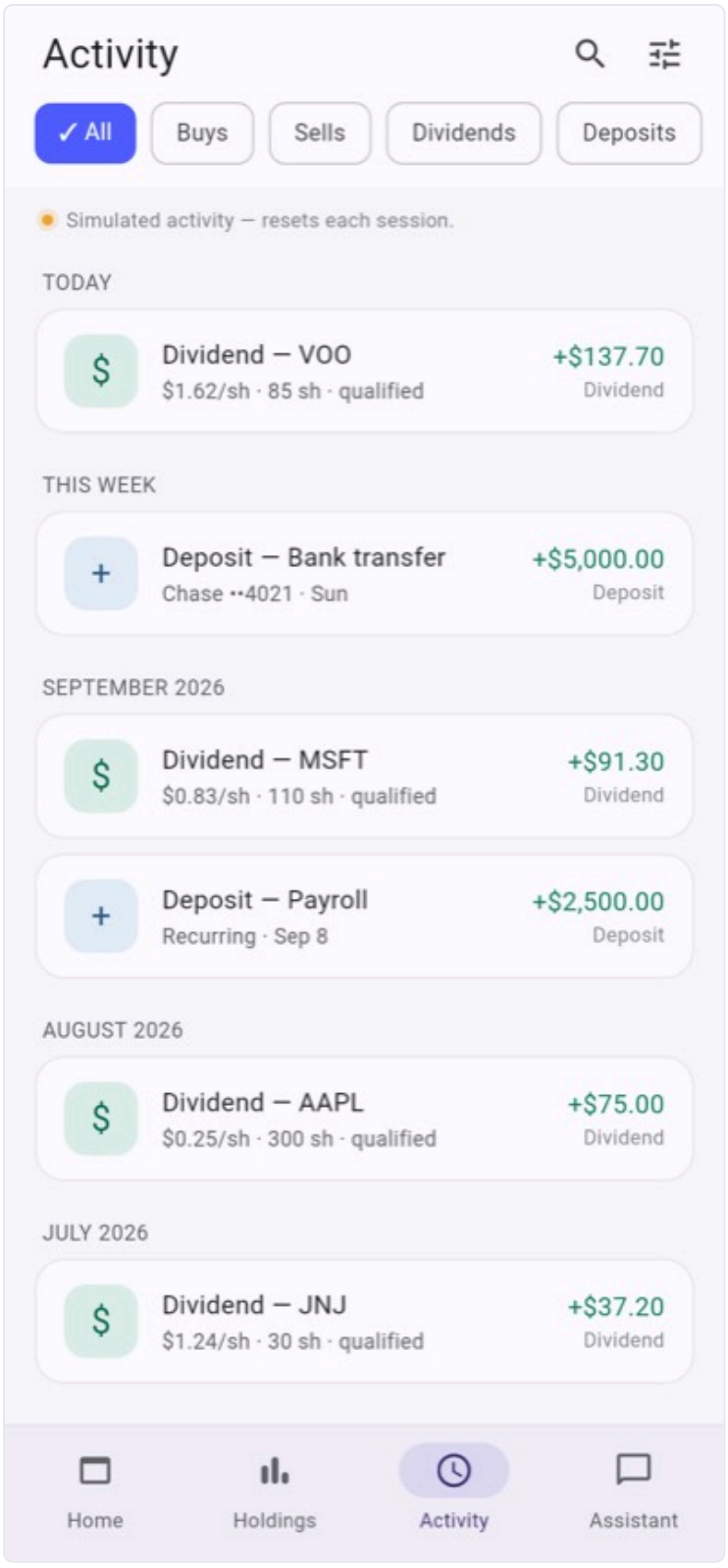


The Holdings tab binds holdings.positions.v1

`activity.timeline.v1` — `ActivityTimeline`

Options: `locale`, `precision`, `currency`, `kind` (`all | buy | sell | dividend | deposit`), `limit`. The filter chips are the payload's `filters[]`; tapping one re-requests the view with `kind=<key>` — the filter is an option, so the client never filters rows itself.

{"view": "activity.timeline.v1", "title": "Activity", "note": "Simulated activity — resets each session.", "kind": "all",
 "filters": [{"key": "all", "label": "All"}, {"key": "buy", "label": "Buys"}, {"key": "sell", "label": "Sells"},
             {"key": "dividend", "label": "Dividends"}, {"key": "deposit", "label": "Deposits"}],
 "groups": [{"label": "TODAY", "entries": [{"key": "txn-0017", "kind": "dividend", "title": "Dividend — VOO",
              "detail": "$1.62/sh · 85 sh · qualified", "amount": {"amount": "137.70…", "display": "+$137.70"},
              "tone": "up", "badge": "Dividend", "realized": null, "term": null}]},
            {"label": "THIS WEEK", "entries": [...]}, {"label": "SEPTEMBER 2026", "entries": [...]}, …]}



The Activity tab binds activity.timeline.v1

`attribution.waterfall.v1` — `AttributionWaterfall`

Shown in full in §4. Options: `fields`, `locale`, `precision`, `currency`. The only view that declares an **action** today (§8).

`attribution.driver.v1` — `DriverDetail`

Options: `locale`, `precision`, `currency`, `symbol`. The card a tapped waterfall bar opens; `symbol` is the option, and the payload echoes it under the same name, so the catalog reports it editable.


```
{"view": "attribution.driver.v1", "symbol": "NVDA", "name": "NVIDIA Corp", "caption": "Top contributor today",
 "contribution": {"amount": "4370.40…", "display": "+$4,370.40"}, "trend": "up",
 "price_move": {"label": "Price move", "percent": {"display": "+4.20%", …}, "amount": {"display": "+$48.56", …}, "trend": "up"},
 "shares": {"label": "Shares", "units": {"units": "90.000000000000000000", "display": "90"}, "price": {"display": "$1,204.66", …}, "caption": "@ $1,204.66"},
 "day_share": {"label": "Of day gain", "percent": {"value": "1.34881395478028998388978390 1", "display": "134.88%"},
              "total": {"display": "+$3,240.18", …}, "caption": "of +$3,240.18"}}
```


`taxlots.table.v1` — `TaxLotTable`

Options: `fields`, `locale`, `precision`, `currency`, `symbol`, `quantity`, `lot_method` (`FIFO | LIFO | HIFO`). `by_term[]` is the short/long split **pre-computed**; `lots[].lots` counts are integers the widget prints, never sums.


```
{"view": "taxlots.table.v1", "title": "Tax lots this sale consumes", "subtitle": "Sell 75 NVDA under FIFO",
 "proceeds":   {"label": "Proceeds",     "amount": {"display": "$90,349.50", …}},
 "cost_basis": {"label": "Cost basis",   "amount": {"display": "$60,000.00", …}},
 "realized":   {"label": "Realised P/L", "amount": {"display": "+$30,349.50", …}},
 "by_term": [{"label": "Short-term", "term": "short", "lots": 1, "realized": {"display": "+$3,669.90", …}},
             {"label": "Long-term",  "term": "long",  "lots": 1, "realized": {"display": "+$26,679.60", …}}],
 "lots": [{"label": "lot-nvda-01", "acquired_on": "2025-09-20", "acquired_on_display": "09/20/2025", "term": "long",
           "units": "60.000000000000000000", "units_display": "60", "cost_basis": {"display": "$45,600.00", …},
           "proceeds": {"display": "$72,279.60", …}, "gain_loss": {"display": "+$26,679.60", …}, "trend": "up", "fully_sold": true}, …]}
```


`scenario.comparison.v1` — `ScenarioComparison`

Options: `locale`, `precision`, `currency`, `symbol`, `quantity`, `methods`. The `spread` block is the comparison already made: `differs`, `highest`, `lowest` are server verdicts, so the widget highlights without comparing.


```
{"view": "scenario.comparison.v1", "title": "Same sale, different lot method",
 "subject": {"label": "NVIDIA Corp", "symbol": "NVDA", "quantity_display": "30", "price": {"display":
```

```
"$1,204.66", …}, "proceeds": {"display": "$36,139.80", …}},
 "columns": [{"method": "FIFO", "label": "FIFO — oldest lots first", "cost_basis": {"display":
"$22,800.00", …},
              "realized_pl": {"display": "+$13,339.80", …}, "short_term_pl": {"display": "$0.00", …},
"long_term_pl": {"display": "+$13,339.80", …},
              "short_term_lots": 0, "long_term_lots": 1, "lots": ["lot-nvda-01"]},
             {"method": "LIFO", …}, {"method": "HIFO", …}],
 "spread": {"amount": {"display": "$6,000.00", …}, "differs": true, "highest": "FIFO", "lowest":
"LIFO"},
 "notes": [{"label": "Before tax", "text": "The realised result and its short/long split, before any
tax rate, fee or wash-sale rule."},
           {"label": "Not advice", "text": "No method is recommended: …"}]}
```


`notes[]` and `disclosure` strings are compliance statements: render them verbatim, never drop them (N-9).

`payoff.curve.v1` — `RiskPayoffChart`

Options: `fields`, `locale`, `precision`, `currency`, `symbol`, `size` (**integer** 1..1000000000, default `500` — a whole number, because a JSON float can no longer be made exact, ADR-E D-5), `size_in` (`units | usd`, default `usd`), `side`, `range_pct` (5..90, default 30), `steps` (1..25, default 12). `markers.break_even.at` and `markers.current.at` are placed by **field path**, not by matching a label.


```
{"view": "payoff.curve.v1", "title": "Payoff across a price range: buy Vanguard S&P 500 ETF",
 "disclosure": "Illustrative linear spot payoff on the demo portfolio's simulated prices: …",
 "markers": {"break_even": {"label": "Break-even", "at": {"amount": "528.00", "display": "$528.00"}},
             "current":    {"label": "Current price", "at": {"amount": "528.00", "display":
"$528.00"}}},
 "totals": [{"label": "Entry price", "amount": {"display": "$528.00", …}}, {"label": "Break-even
price", …}, {"label": "Notional", "amount": {"display": "$500.00", …}}],
 "series": [{"name": "Profit / loss", "points": [
   {"label": "$369.60", "at": {"amount": "369.600", "display": "$369.60"}, "value": {"amount":
"-150.00…", "display": "-$150.00"}},
   … 25 points …,
   {"label": "$686.40", "at": {"amount": "686.400", "display": "$686.40"}, "value": {"amount":
"150.00…", "display": "$150.00"}}]}],
 "annotations": [...]}
```


`points[].at` is the **x** coordinate as a pair: a client given only the `label` would have to parse `"$369.60"` back into a number to place the point, which is the arithmetic ADR-K D-5 removes.

`order.ticket.v1` — `OrderTicket`

Options: `locale`, `precision`, `currency`, `symbol`, `side`, `size` (string, `^[0-9]{1,9}(\.[0-9]{1,8})?$`), `size_in` (`units | usd`). The first interactive projection: `inputs[]` publishes one descriptor per editable option with its **current value and its bound**, and the payload echoes `symbol`, `side`, `size`, `size_in` under the option names.

```
…?view=order.ticket.v1&symbol=AAPL&size=500&size_in=usd
```


```
{"view": "order.ticket.v1", "title": "Proposed order", "symbol": "AAPL", "name": "Apple Inc.", "side":
"buy",
 "size": "500", "size_in": "usd",
 "quantity":    {"units": "2.144266232095376962003602367", "display": "2.14426623"},
 "entry_price": {"amount": "233.18", "display": "$233.18"},
 "notional":    {"label": "Order value", "amount": {"amount": "499.999999999999999999999999",
"display": "$500.00"}},
 "inputs": [{"option": "symbol",  "label": "Asset",   "control": "choice", "value": "AAPL", "choices":
["AAPL", "AGG", …, "XOM"], "pattern": null, "hint": null},
            {"option": "side",    "label": "Side",    "control": "choice", "value": "buy",  "choices":
["buy", "sell"], "pattern": null, "hint": null},
            {"option": "size",    "label": "Size",    "control": "text",   "value": "500",  "choices":
[], "pattern": "^[0-9]{1,9}(\\.[0-9]{1,8})?$", "hint": "Up to 8 decimal places"},
            {"option": "size_in", "label": "Size in", "control": "choice", "value": "usd",  "choices":
["units", "usd"], "pattern": null, "hint": null}],
 "effect": "recalculate",
 "disclosure": "Illustrative ticket. This showcase has no broker and no execution: changing a field
re-renders these figures, and nothing is ever sent, placed or settled."}
```

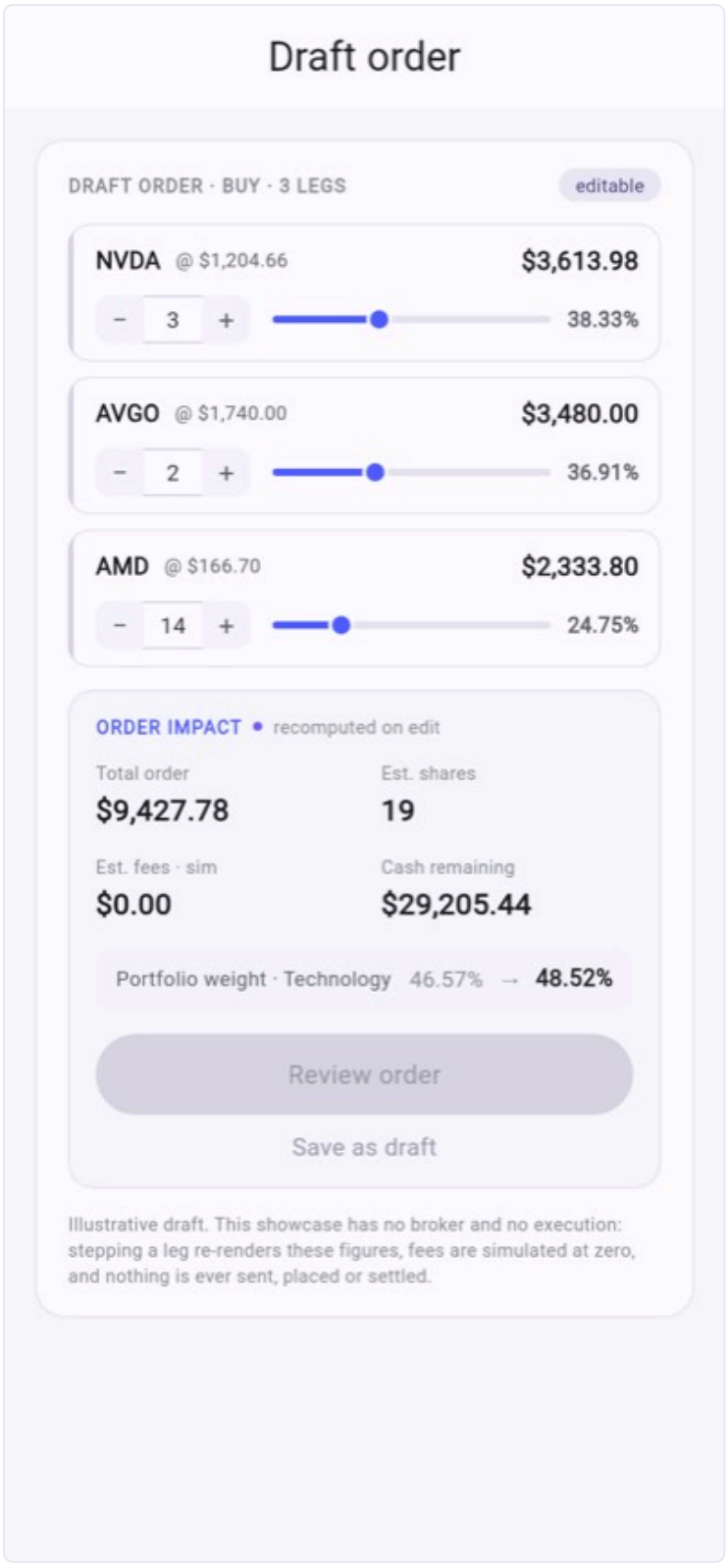

The division `500 / 233.18` happened **server-side**: `quantity` is an output of the render, which is why it is *not* named after an option and *not* editable.

`order.ticket.v2` — `MultiLegOrderTicket`

Options: `locale`, `precision`, `currency`, `legs` (one bounded string, `NVDA:3,AVGO:2,AMD:14`), `side`. Each line carries `increment` **and** `decrement`**: the whole** `legs` **value the stepper sends when tapped**, composed by the backend — so the client never adds one to a share count (N-2). The edit round-trip is walked in `schema-mutations.md` §3.


```
{"view": "order.ticket.v2", "title": "Draft order · Buy · 3 legs", "badge": "editable", "side": "buy",
 "legs": "NVDA:3,AVGO:2,AMD:14",
 "lines": [{"key": "NVDA", "symbol": "NVDA", "name": "NVIDIA Corp", "price": {"display": "$1,204.66",
…},
            "quantity": {"units": "3", "display": "3"}, "notional": {"display": "$3,613.98", …},
"weight": {"display": "38.33%", …},
            "increment": "NVDA:4,AVGO:2,AMD:14", "decrement": "NVDA:2,AVGO:2,AMD:14"},
           {"key": "AVGO", …, "increment": "NVDA:3,AVGO:3,AMD:14", "decrement":
"NVDA:3,AVGO:1,AMD:14"},
           {"key": "AMD",  …, "increment": "NVDA:3,AVGO:2,AMD:15", "decrement":
"NVDA:3,AVGO:2,AMD:13"}],
 "impact": {"total": {"label": "Total order", "amount": {"display": "$9,427.78", …}},
            "units": {"label": "Est. shares", "units": {"units": "19", "display": "19"}},
            "fees":  {"label": "Est. fees · sim", "amount": {"display": "$0.00", …}},
```

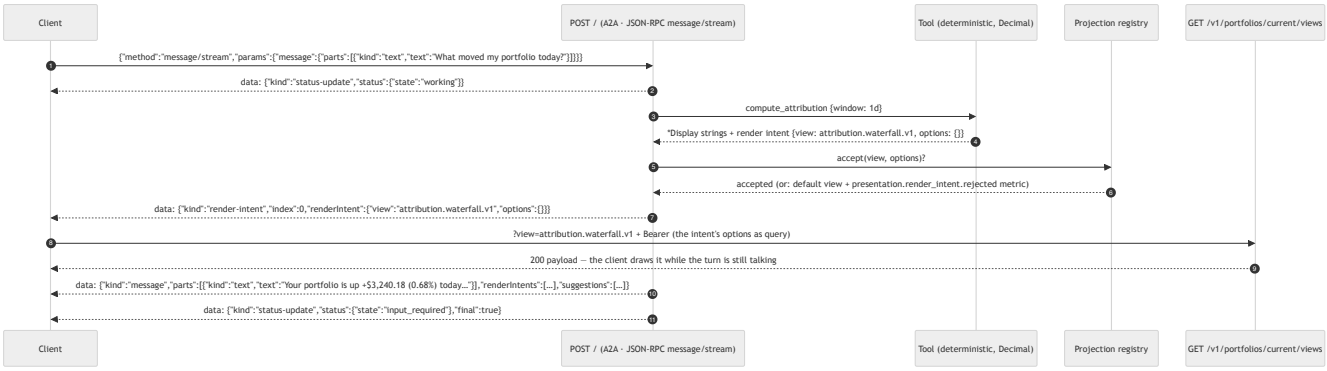
```
            "cash_remaining": {"label": "Cash remaining", "amount": {"display": "$29,205.44", …}},
            "weight_shifts": [{"label": "Portfolio weight · Technology", "key": "technology",
                               "before": {"display": "46.57%", …}, "after": {"display": "48.52%",
…}}]},
 "constraints": {"max_cash": {"label": "Available cash", "amount": {"display": "$38,633.22", …}},
"breach": "none", "shortfall": null, "message": null},
 "disclosure": "Illustrative draft. This showcase has no broker and no execution: …", "effect":
"recalculate"}
```



order.ticket.v2 bound by the client: steppers, weights and the impact panel are all payload values

8. The same charts inside an agent turn (A2UI)

The assistant does not return a payload. It returns a **render intent** — `{view, options}` picked from the registry — streamed the moment the tool that produced it returned, and the client fetches the view through the exact read path of §4. The full picture, including the model loop and the tool registry, is `docs/diagrams/05-agent-turn.md`; this is the client's slice of it:



The stream, exactly as captured (this stack runs without a Gemini key, so the **scripted** assistant of spec 0033 answers — same tools, same registry, same event order; only the party choosing the tool call is a table instead of a model):

```
curl -s -N -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" localhost:8000/ -d '{
  "jsonrpc":"2.0","id":"1","method":"message/stream",
  "params":{"message":{"role":"user","messageId":"m-1",
            "parts":[{"kind":"text","text":"What moved my portfolio today?"}]}}}'
```

```
data: {"kind":"status-update","taskId":"202ef0b9-…","contextId":null,"status":
{"state":"working"},"final":false}

data: {"kind":"render-intent","taskId":"202ef0b9-…","contextId":null,"index":0,"renderIntent":
{"view":"attribution.waterfall.v1","options":{}}}

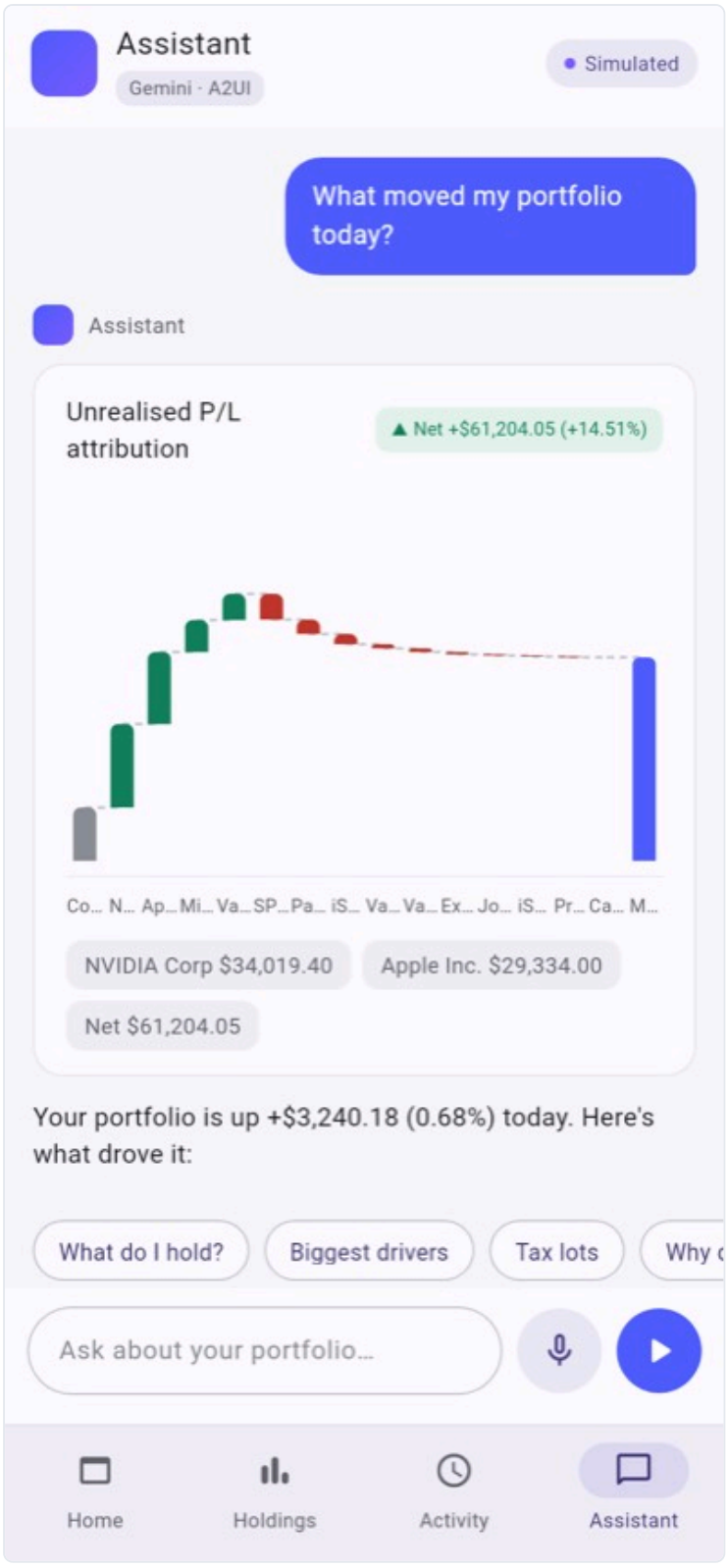
data: {"kind":"message","taskId":"202ef0b9-…","contextId":null,"role":"agent",
       "parts":[{"kind":"text","text":"Your portfolio is up +$3,240.18 (0.68%) today. Here's what
drove it:"}],
       "renderIntents":[{"view":"attribution.waterfall.v1","options":{}}],
       "suggestions":[{"key":"allocation","label":"What do I hold?"},{"key":"biggest-
drivers","label":"Biggest drivers"},
                      {"key":"tax-lots","label":"Tax lots"},{"key":"why-did-i-move","label":"Why did I
move?"}]}

data: {"kind":"status-update","taskId":"202ef0b9-…","contextId":null,"status":
{"state":"input_required"},"final":true}
```

What a client does with each event:

`kind`	**Do**
`status-update` `working`	show the turn as in progress

`kind`	**Do**
`render-intent`	`GET /v1/portfolios/current/views?view=<renderIntent.view>&<renderIntent.options as query>` and render the payload **now** — first pixel before the sentence arrives (ADR-K D-19)
`message`	show `parts[].text` verbatim (its figures were copied from tool output, never computed by the model); `renderIntents` repeats every intent so a client that ignored the incremental events still has the full list; `suggestions` are the chips it may offer next
`status-update` `final: true`	`input_required` = the turn ended and awaits the user; `completed` / `failed` as named

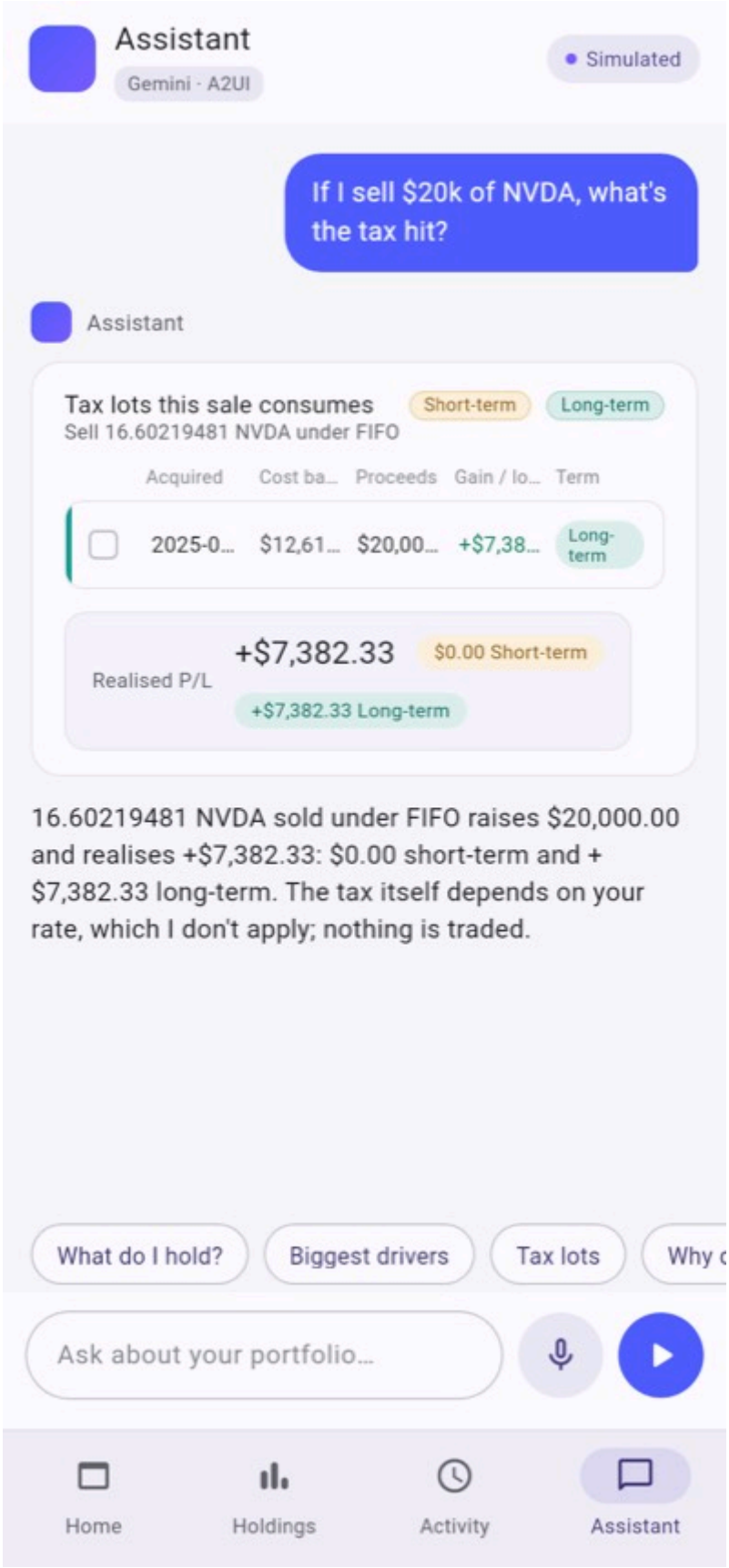


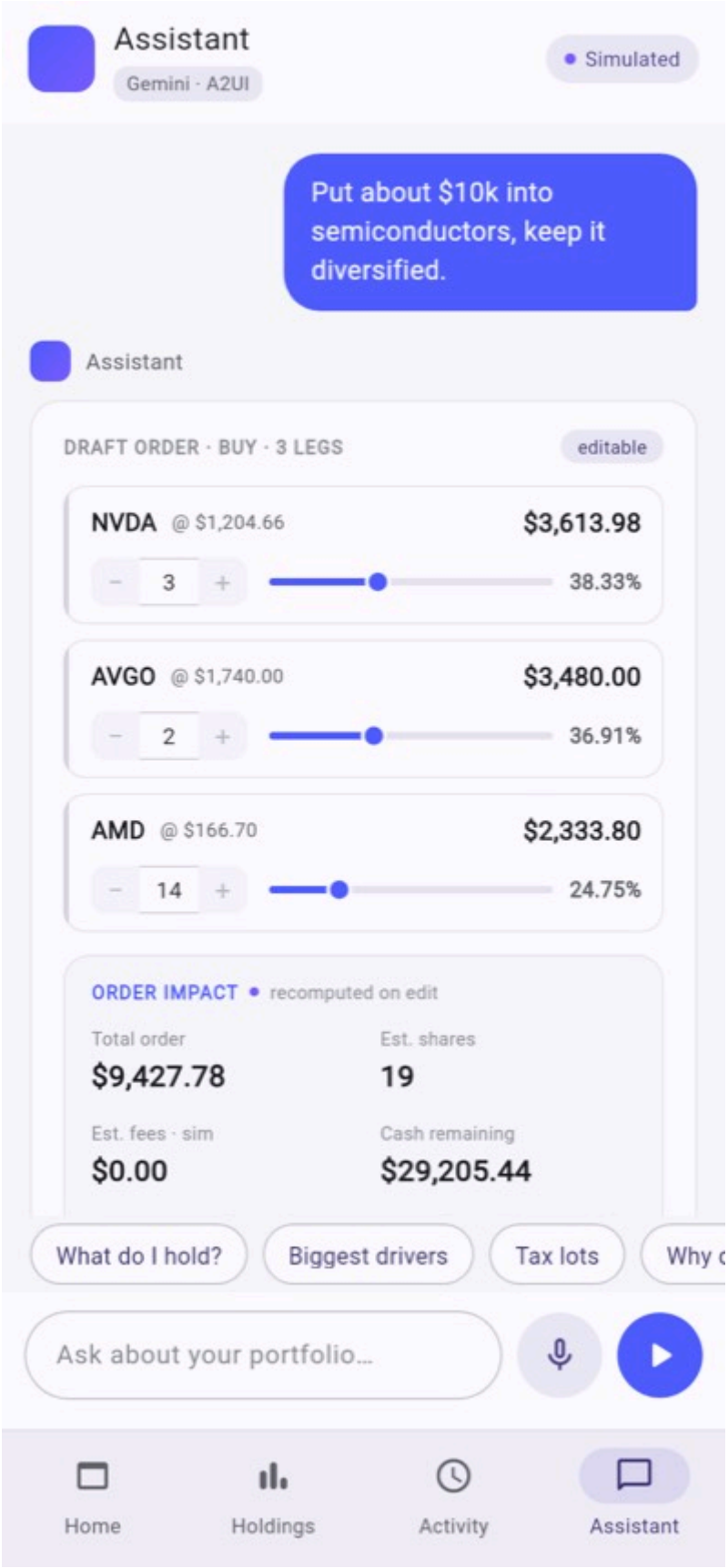
The Assistant tab: the render-intent fetched as attribution.waterfall.v1 and drawn before the sentence

The three design flows, as the scripted assistant answers them on this build (the chart in each case is the *tool's* choice, judged by the registry — the script aims nothing):

Prompt	Tool	Render intent
What moved my portfolio today?	`compute_attribution {window: 1d}`	`attribution.waterfall.v1`

Prompt	Tool	Render intent
If I sell $20k of NVDA, what's the tax hit?	`simulate_sale {symbol: NVDA, amount_usd: "20000", lot_method: FIFO}`	`taxlots.table.v1`
Put about $10k into semiconductors, keep it diversified.	`draft_order {amount: "10000", basket: semiconductors}`	`order.ticket.v2` with the same `legs`


Assistant
Gemini · A2UI
Simulated
If I sell $20k of NVDA, what's the tax hit?
Assistant
Tax lots this sale consumes
Short-term
Long-term
Sell 16.60219481 NVDA under FIFO
Acquired
Cost ba...
Proceeds
Gain / lo...
Term
2025-0...
$12,61...
$20,00...
+$7,38...
Long-term
Realised P/L
+$7,382.33
$0.00 Short-term
+$7,382.33 Long-term
16.60219481 NVDA sold under FIFO raises $20,000.00 and realises +$7,382.33: $0.00 short-term and + $7,382.33 long-term. The tax itself depends on your rate, which I don't apply; nothing is traded.
What do I hold?
Biggest drivers
Tax lots
Why
Ask about your portfolio...
Home
Holdings
Activity
Assistant



8.1 Sending something back: chips and taps are declared actions

A surface can send **an action id it did not invent, plus scalars drawn from bounds the payload published** — never a sentence. It travels as A2A's own `data` part; the prompt the turn receives is composed server-side

(spec 0022). The chips in `suggestions[]` are `ask.suggested.v1` with the chip's `key`; a tapped waterfall bar is `drill.attribution.v1` with the bar's `buckets[].key`:

```
# A chip
curl -s -N -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" localhost:8000/ -d '{
  "jsonrpc":"2.0","id":"2","method":"message/stream",
  "params":{"message":{"role":"user","messageId":"m-2",
            "parts":[{"kind":"data","data":{"action":"ask.suggested.v1","args":
{"suggestion":"allocation"}}}]}}}'
# → render-intent {"view":"holdings.allocation.v1","options":{"top_n":14}}
# → message "Your portfolio is worth $482,915.20, +$61,204.05 (14.51%) on what it cost. Here is how it
is allocated, largest first:"

# A tapped bar
… "parts":[{"kind":"data","data":{"action":"drill.attribution.v1","args":{"driver":"NVDA"}}}] …
# → render-intent {"view":"attribution.driver.v1","options":{"symbol":"NVDA"}}
# → message "NVIDIA Corp (NVDA) contributed +$4,370.40 today: its price moved +4.20% (+$48.56) on 90
shares at $1,204.66, 134.88% of the day's +$3,240.18."
```

Note the second intent carries an **option** (`symbol: NVDA`) the tool filled: the client fetches `…?view=attribution.driver.v1&symbol=NVDA`, exactly as it would for any option. An unknown action or an out-of-range argument is a JSON-RPC `-32602` at HTTP 200 and the stream never opens; unlike a bad render intent there is no default to fall back to.

9. Checklist for a new client

□ Read `GET /v1/views` once; keep `id`, `component`, `options_schema`, `fields`.

□ Bind every figure from a `display` string; use `amount` / `value` / `units` for coordinates only.

□ Render rows, notes and disclosures in arrival order and verbatim.

□ Send only catalog ids and declared options; treat 422/404 as contract errors, not retries.

□ Echo `ETag` back; treat 304 as the cached body.

□ Batch same-portfolio views with repeated `view=` (≤ 4); use `POST /v1/view-renders` (≤ 8) for per-view options or several portfolios.

□ In a turn, fetch each `render-intent` as it arrives; send taps back as `data` parts with published values.

□ A view id with no registered component is a placeholder, not an error — the catalog grows on its own schedule.

Schema mutations — how a chart's shape is allowed to change

Audience: anyone adding a chart, changing one, or writing a client that must survive both. **Companion:** `feeding-charts.md`, which explains how the payloads described here are fetched. **Source of truth:** ADR-K (D-3 declared options, D-9 versioning, D-11/D-12 render intents, D-14/D-15 the seam), `.claude/rules/backend/presentation.md` §6 (the checklist a new chart follows) and the specs each section cites.

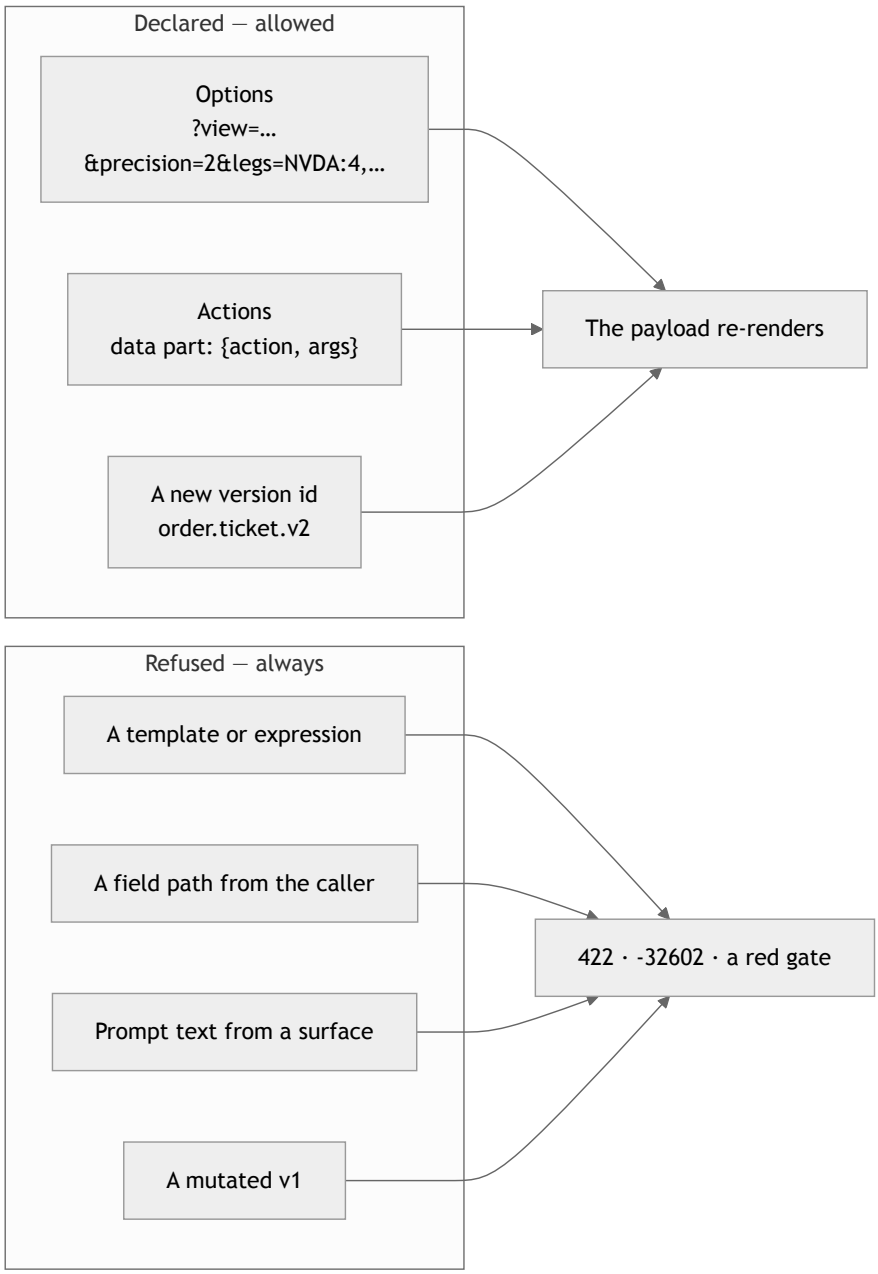
Everything below was verified against a running backend on 2026-09-22 (`local+b7e78bf`); commands and responses are as they came back.

0. The short version

A payload may vary in exactly **two** ways, and change in exactly **one**:

	Mechanism	Bound
Varies	a **declared option** the caller fills	the projection's own pydantic model — bounded, defaulted, described. Anything undeclared or out of range is a **422**
Varies	a **declared action** the surface sends back	a registered action id plus scalars from a published bound. Anything else is a JSON-RPC `-32602`
Changes	a **new version id** — `<component>.<variant>.v<N+1>`	the old id keeps serving, byte for byte, with its digest unmoved

There is no third mechanism. No free-form template, no expression language, no field-path DSL, no client-supplied prompt text. That is not an omission; ADR-K lists the interpreter it declined and why.



1. A versioned id is a promise, and `v1` and `v2` prove it

An id is `<component>.<variant>.v<N>`. Once a client binds to `attribution.waterfall.v1`, **that shape is fixed** (ADR-K D-9). A breaking change arrives as a new id; it never arrives as a mutation.

The worked example is in the repository right now. The design for the multi-leg draft order could have been argued as "the order ticket, with legs". Spec 0024 §3 ruled that it is not a backwards-compatible addition to `order.ticket.v1` but **a second contract**, and 0031 built it as one. Both are served today:

```
$ curl -s -H "Authorization: Bearer $TOKEN" localhost:8000/v1/views \
    | python3 -c 'import sys,json;[print(v["id"],"→",v["component"]) for v in json.load(sys.stdin) if "order" in v["id"]]'
```

```
order.ticket.v1 → OrderTicket
order.ticket.v2 → MultiLegOrderTicket
```

	`order.ticket.v1`	`order.ticket.v2`
Component	`OrderTicket`	`MultiLegOrderTicket`
Digest	`d0daddcb8960`	`e6c713d6a144`
Fields	28	64
Own options	`symbol`, `side`, `size`, `size_in`	`legs`, `side`
Shape of the edit	an `inputs[]` block, one descriptor per editable option	`legs` echoed, plus `increment` / `decrement` per line
Extras	—	`impact` panel, `constraints` with a cash breach

Spec 0031 §2 states the rule as an outcome, not an intention: v1 is "*unchanged byte for byte — its module, payload, snapshot entry and digest* `d0daddcb8960` *are the same.*" A digest is what makes that checkable instead of promised (§9).

Note also what v2 did **not** get: the component name. `OrderTicket` belongs to v1 and stays there; v2 is `MultiLegOrderTicket`. A version bump that reuses a component name would leave the client unable to tell the two widgets apart.

When is a change a new version rather than an option?

The change	It is
A new optional field the old client can ignore	an addition to `vN`, and the digest moves — see §9
A field removed, renamed, retyped, or re-bounded downward	a **new version**
A different arrangement of the same data for the same question	an **option** (`group_by`, `sort`, `range`, `lot_method`)
A different question, or a different component	a **new projection** — a new id, not a new version

Removing an id at all needs a deprecation note in `CHANGELOG.md` and a release cycle (ADR-K D-9). Nothing has been removed yet.

---

2. Options are the only knob, and they are bounded

Every projection publishes a pydantic options model: each field bounded (`ge` / `le` / `max_length` / `enum` / `const` / `pattern`), defaulted, described and exemplified, with `additionalProperties: false` on the schema. The schema in `GET /v1/views` **is** the schema that validates, because both come from the same model.

```
"legs": {
  "type": "string",
  "default": "NVDA:3,AVGO:2,AMD:14",
```

```
  "maxLength": 89,
  "pattern": "^[A-Z]{2,10}:[0-9]{1,6}(,[A-Z]{2,10}:[0-9]{1,6}){0,4}$",
  "description": "The legs of the draft as `TICKER:SHARES` pairs separated by commas, one to five of them, whole shares, each ticker from the market reference and named once. Every rendered leg carries the value a `+` or `−` sends back, so a stepper edit is this option re-sent with one number moved.",
  "examples": ["NVDA:3,AVGO:2,AMD:14", "NVDA:4,AVGO:2,AMD:14"]
}
```

Both callers — the client's `?view=` and the model's render intent — are judged by that one model, in `presentation/resolve.py`:

```
$ …/views?view=attribution.waterfall.v1&legs=NVDA:3          → 422
  "detail": "view 'attribution.waterfall.v1' rejected option(s) ['legs']"

$ …/views?view=attribution.waterfall.v1&fields=nope          → 422
  "detail": "unknown section(s) ['nope']; this view offers ['buckets', 'annotations']"

$ …/views?view=attribution.waterfall.v1&precision=99          → 422
$ …/views?view=attribution.waterfall.v1&sparkle=1             → 422
  "detail": "1 request field(s) failed validation"
```

Two details worth copying into any new options model:

- **The rejection names the option, never its value.** An echoed input is how a rejection becomes a reflection vector.
- **Unknown is rejected, not ignored.** A best-effort render of an option the server did not understand is a payload nobody can reason about.

---

3. An editable field is a payload field named exactly as the option that sets it

This is the whole interactivity declaration, and it is a **naming rule**, not a list. From `.claude/rules/backend/presentation.md` §4:

> An editable field is a payload field named exactly as the option that sets it. That naming *is* the interactivity declaration; a component with an input that does not round-trip through a declared option breaks D-3.

The contract generator applies exactly that test when it fills the `input` column, which is why the answer is derived rather than curated:

```
{"path": "side", "binding": "enum", "state": "variant",
 "input": "input · one of 'buy', 'sell'"}
{"path": "legs", "binding": "text", "state": null,
 "input": "input · string matching ^[A-Z]{2,10}:[0-9]{1,6}(,[A-Z]{2,10}:[0-9]{1,6}){0,4}$"}
```

`order.ticket.v2` has 64 fields and exactly those two are editable, because exactly those two are named after options. `lines[].notional` is an **output** of the render — no option is called `notional`, so nothing offers to edit it. The same test explains `attribution.driver.v1`: it echoes `symbol` under the option's own name, so the catalog reports `symbol` editable and nothing else.

The three shapes this takes today

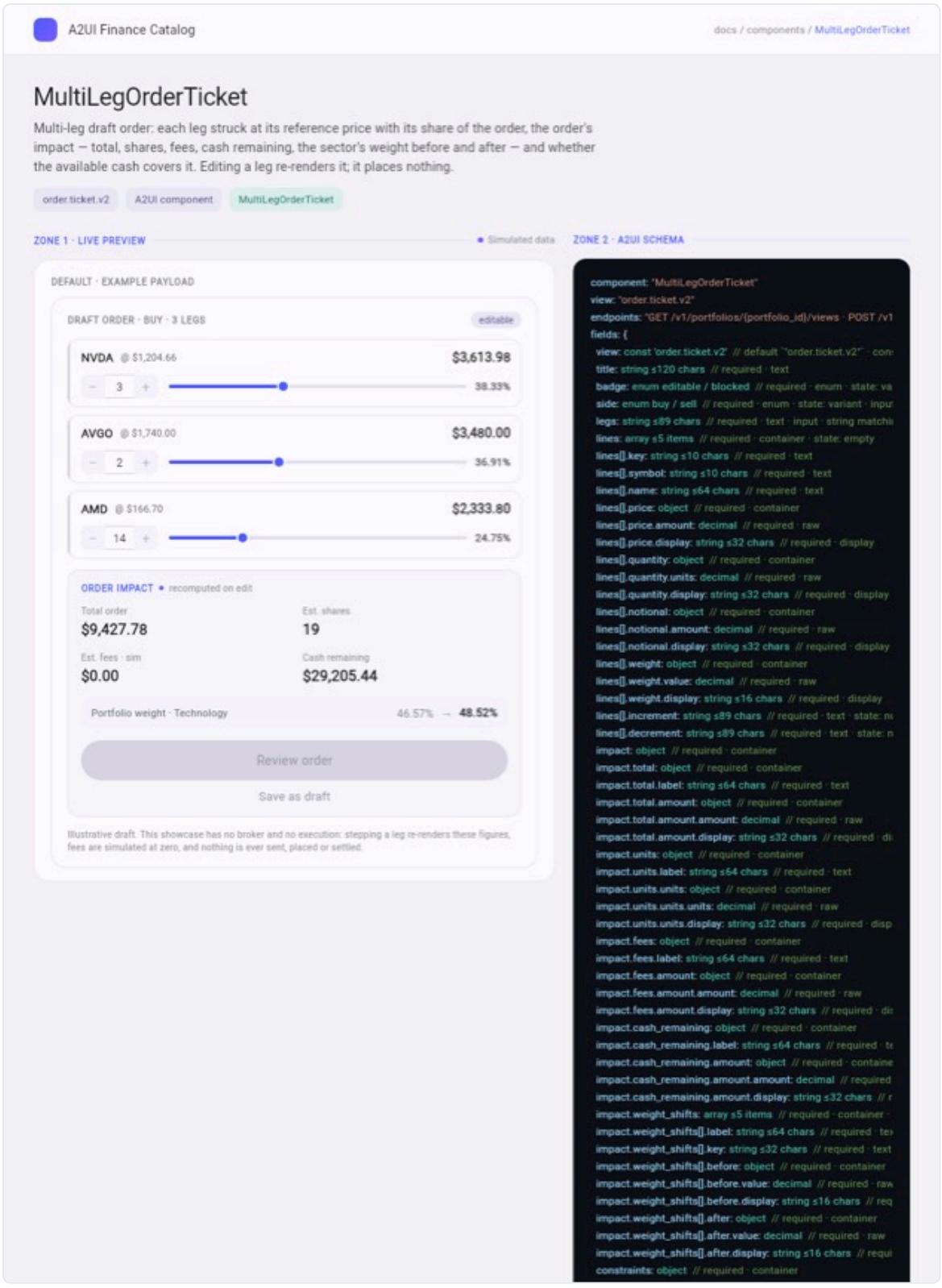
(a) `order.ticket.v1` — an `inputs[]` descriptor block. One entry per editable option, carrying its current value and its bound, so the widget renders the control from the payload rather than from a hardcoded form:

```
"inputs": [
  {"option": "symbol",  "label": "Asset",   "control": "choice", "value": "NVDA",
   "choices": ["AAPL", "AGG", "AMD", "…", "XOM"], "pattern": null, "hint": null},
  {"option": "side",    "label": "Side",    "control": "choice", "value": "buy",
   "choices": ["buy", "sell"], "pattern": null, "hint": null},
  {"option": "size",    "label": "Size",    "control": "text",   "value": "3",
   "choices": [], "pattern": "^[0-9]{1,9}(\\.[0-9]{1,8})?$", "hint": "Up to 8 decimal places"}
]
```

(b) `order.ticket.v2` — the next value, precomputed per leg. The design's stepper adds one to a share count. Adding one is arithmetic, and the client may not perform arithmetic even once (N-2). So the payload hands it the answer: every line carries the **whole `legs` value** a `+` or `−` sends back.

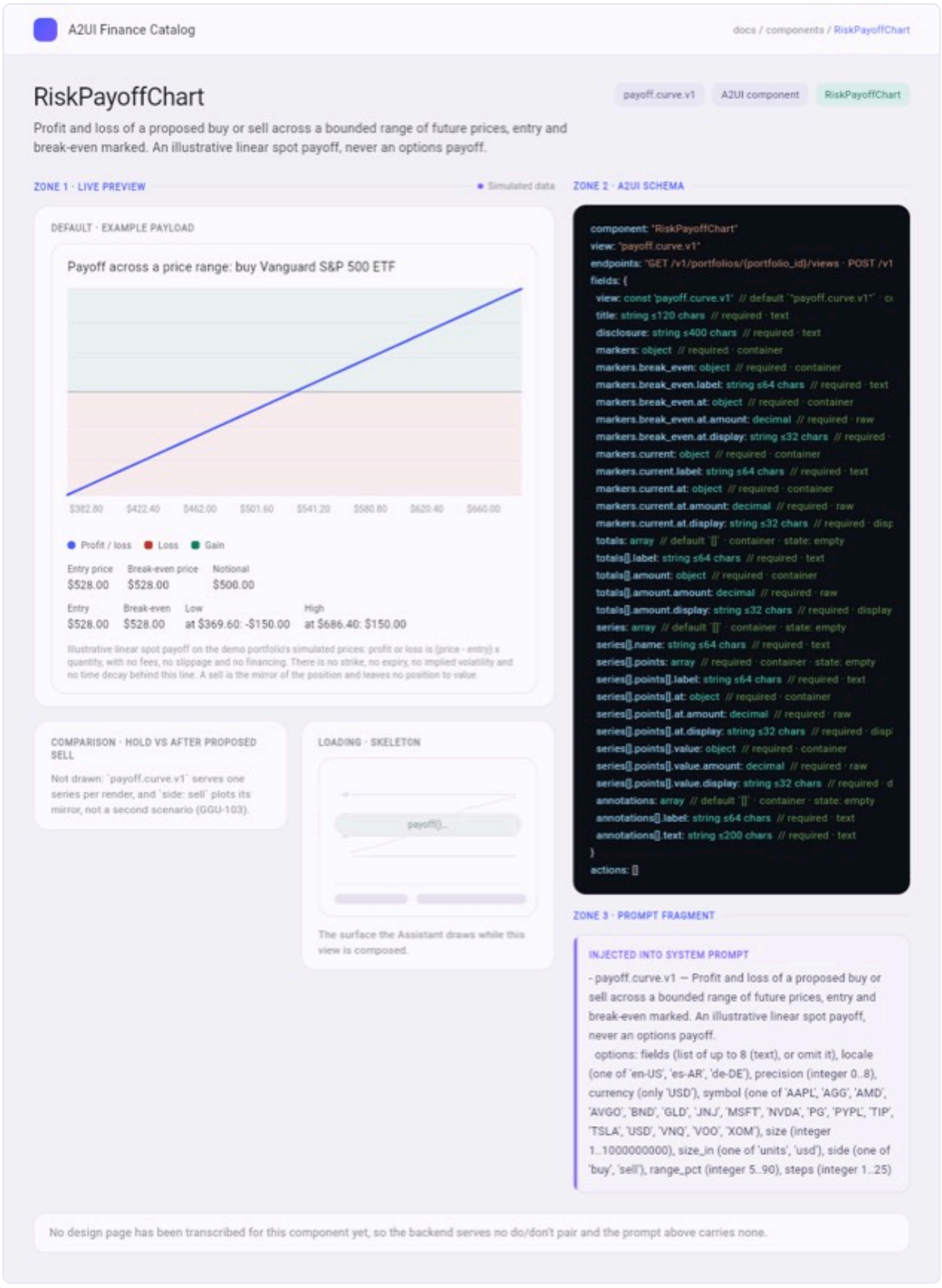
```
"legs": "NVDA:3,AVGO:2,AMD:14",
"lines": [
  {"key": "NVDA", "quantity": {"units": "3",  "display": "3"},
   "increment": "NVDA:4,AVGO:2,AMD:14", "decrement": "NVDA:2,AVGO:2,AMD:14"},
  {"key": "AVGO", "quantity": {"units": "2",  "display": "2"},
   "increment": "NVDA:3,AVGO:3,AMD:14", "decrement": "NVDA:3,AVGO:1,AMD:14"},
  {"key": "AMD",  "quantity": {"units": "14", "display": "14"},
   "increment": "NVDA:3,AVGO:2,AMD:15", "decrement": "NVDA:3,AVGO:2,AMD:13"}
]
```

A tap is not an event the client interprets. It is a string it copies into the next request.



The MultiLegOrderTicket catalog page: the default draft, the state after a stepper edit, and the over-cash breach, beside the A2UI schema block that lists `legs` and `side` as the only inputs

(c) `payoff.curve.v1` — a scalar echoed under its option name. `size`, `symbol`, `side`, `range_pct` and `steps` are options; the payload echoes the ones it is asked to make adjustable, and the curve re-renders. The same mechanism, without a descriptor block, because the chart has no form.



The RiskPayoffChart catalog page: `size`, `side`, `range_pct` and `steps` are options, and the curve is redrawn server-side for each

The round trip, actually run

Before — the default three legs:

```
$ curl -s -H "Authorization: Bearer $TOKEN" \
    "localhost:8000/v1/portfolios/current/views?
```

```
view=order.ticket.v2&legs=NVDA:3,AVGO:2,AMD:14&side=buy"

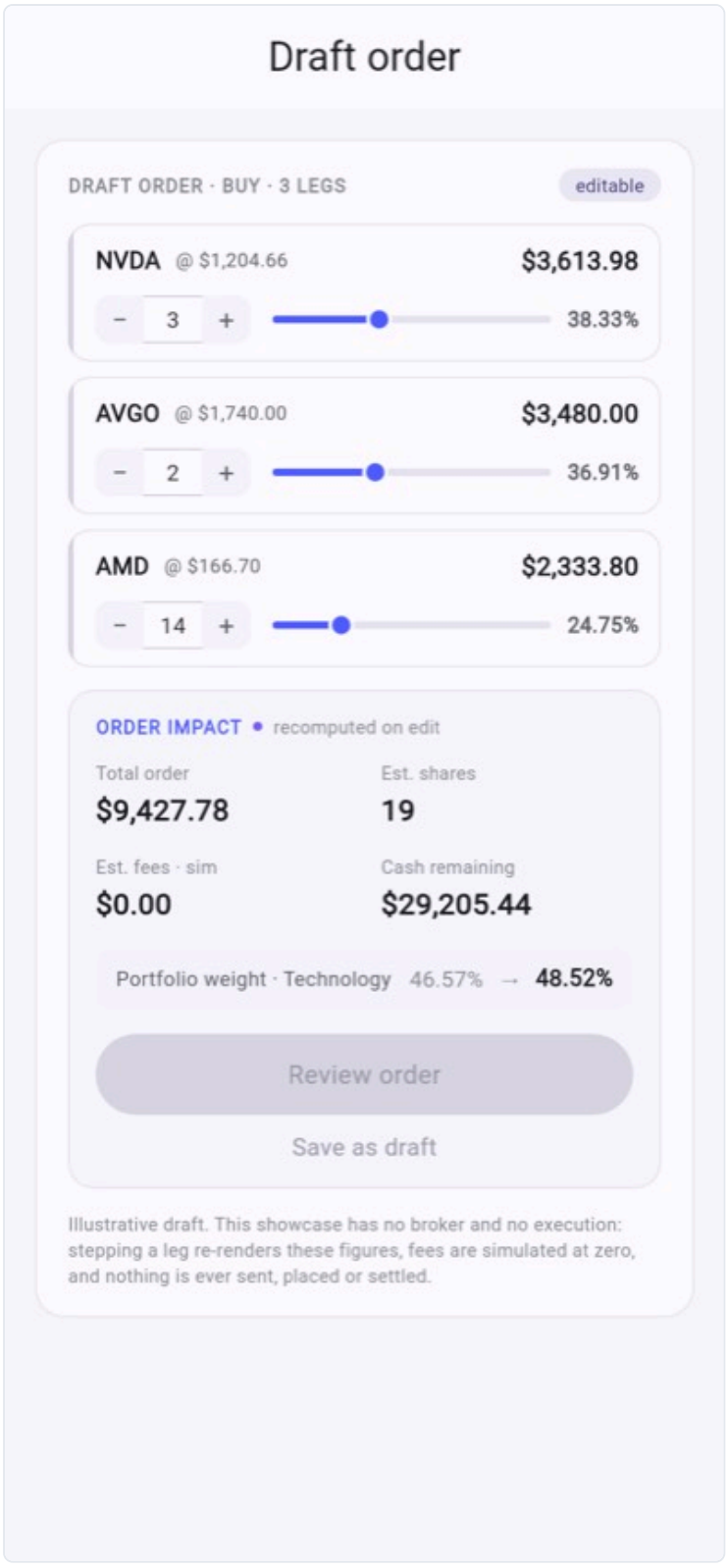
title  : Draft order · Buy · 3 legs      legs: NVDA:3,AVGO:2,AMD:14
NVDA  3   $3,613.98  38.33%   +: NVDA:4,AVGO:2,AMD:14   −: NVDA:2,AVGO:2,AMD:14
AVGO  2   $3,480.00  36.91%   +: NVDA:3,AVGO:3,AMD:14   −: NVDA:3,AVGO:1,AMD:14
AMD   14  $2,333.80  24.75%   +: NVDA:3,AVGO:2,AMD:15   −: NVDA:3,AVGO:2,AMD:13
total : $9,427.78   cash remaining: $29,205.44   breach: none
```

The user taps `+` on NVDA. The client sends back that line's `increment` **verbatim** as the `legs` option — it composes nothing:

```
$ curl -s -H "Authorization: Bearer $TOKEN" \
    "localhost:8000/v1/portfolios/current/views?
view=order.ticket.v2&legs=NVDA:4,AVGO:2,AMD:14&side=buy"

title  : Draft order · Buy · 3 legs      legs: NVDA:4,AVGO:2,AMD:14
NVDA  4   $4,818.64  45.32%   +: NVDA:5,AVGO:2,AMD:14   −: NVDA:3,AVGO:2,AMD:14
AVGO  2   $3,480.00  32.73%   +: NVDA:4,AVGO:3,AMD:14   −: NVDA:4,AVGO:1,AMD:14
AMD   14  $2,333.80  21.95%   +: NVDA:4,AVGO:2,AMD:15   −: NVDA:4,AVGO:2,AMD:13
total : $10,632.44  cash remaining: $28,000.78  breach: none
```

Every one of those figures moved server-side: the notional, all three weights, the total, the cash remaining, and the next `increment` / `decrement` pair on every line. The client changed one query parameter and redrew.



order.ticket.v2 as the client binds it: steppers, weights and the impact panel are all payload values

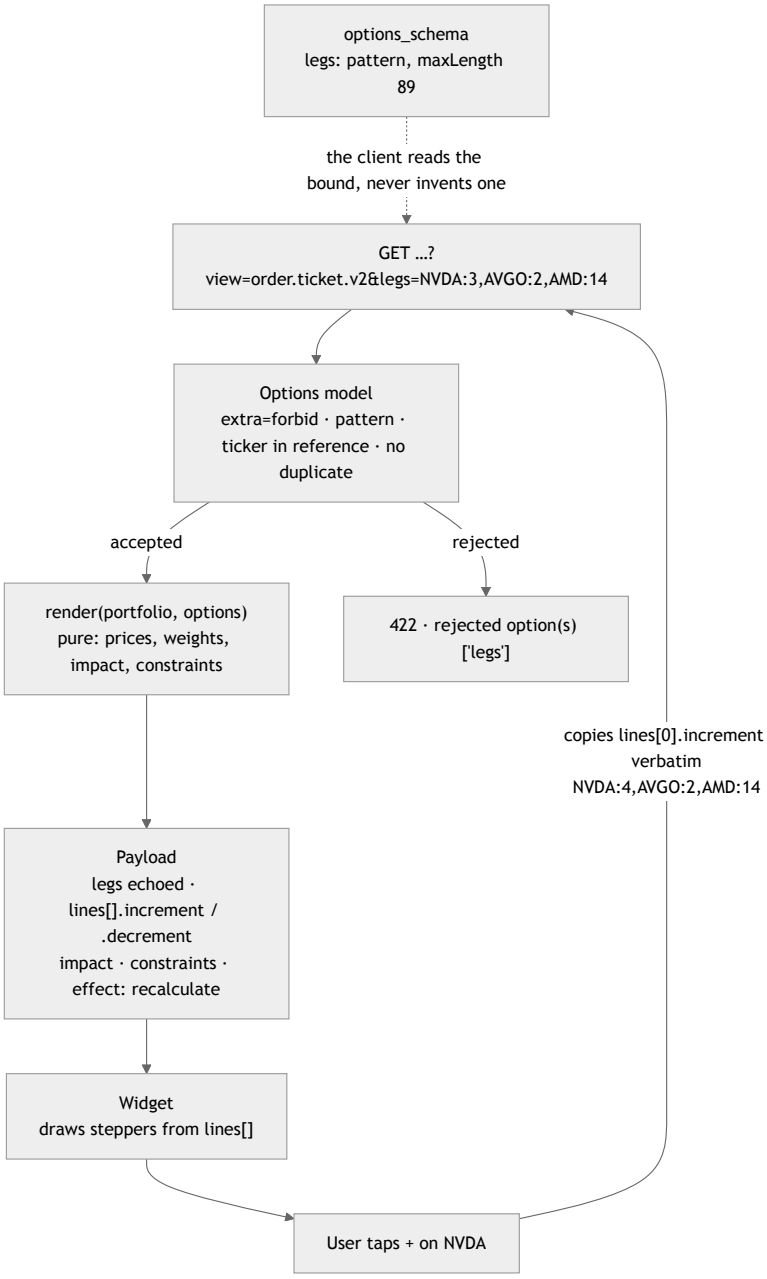
The bounds are enforced on the way back too. A leg the portfolio's cash cannot cover is not a client-side warning; it is a payload state:

```
$ …?view=order.ticket.v2&legs=NVDA:900&side=buy
"constraints": {
  "max_cash":  {"label": "Available cash", "amount": {"display": "$38,633.22", …}},
  "breach":    "over_cash",
```

```
  "shortfall": {"amount": "1045560.78…", "display": "$1,045,560.78"},
  "message":   "Exceeds available cash — reduce order by $1,045,560.78."
}
```

`breach` is an enum the widget switches on; `message` is a sentence it prints verbatim. It computed neither.

The round trip, drawn



---

4. The model picks from the same registry, and a bad guess is survivable

The agent does not emit a payload. It emits a **render intent** — `{view, options}` — and that intent meets the *same* registry and the *same* options models the client's `?view=` meets (ADR-K D-11).

What it may not do is invent. `RenderIntentService.accept` is the one judgement seat:

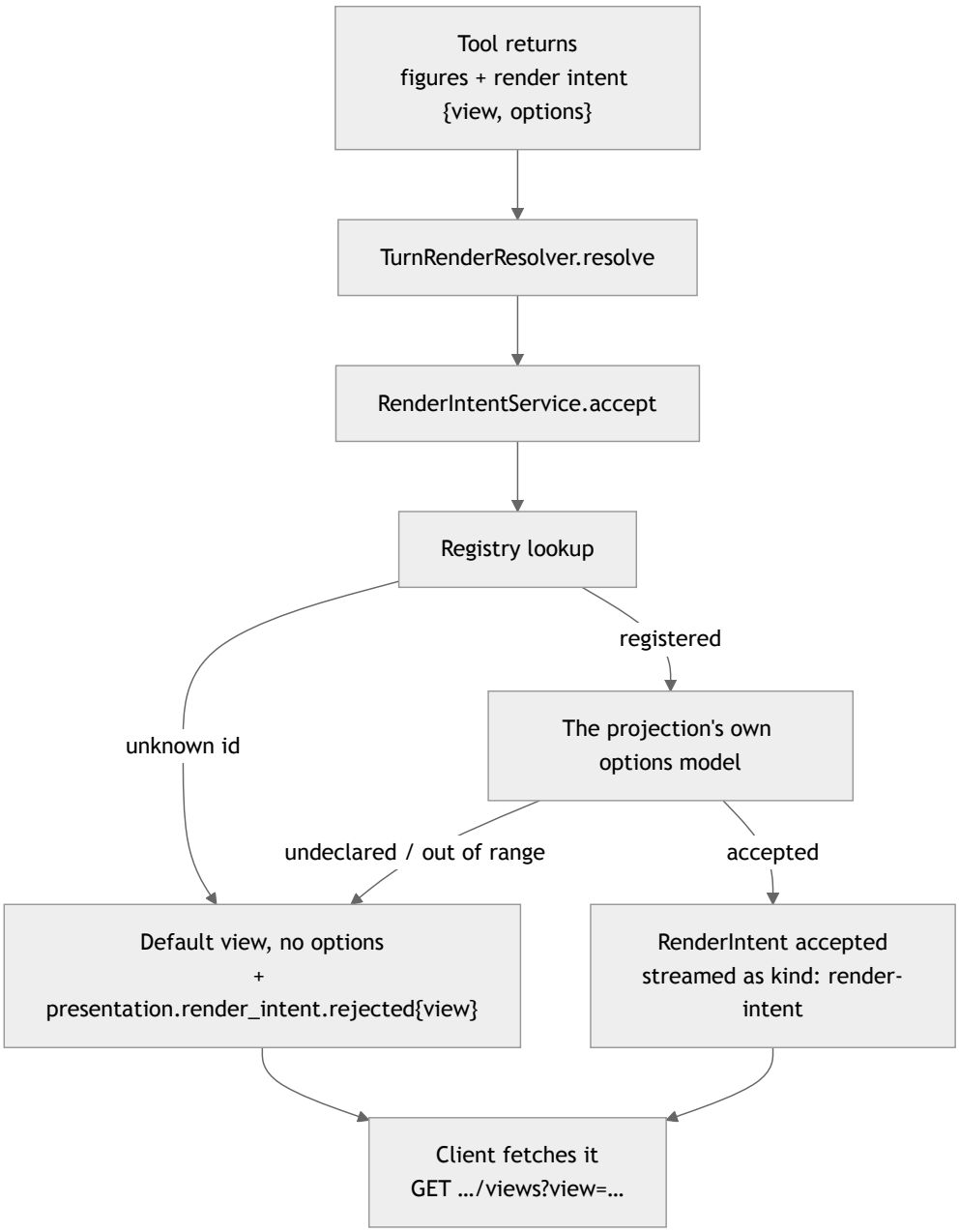
```
try:
    self._views.validate(view=view, options=options)
except AppError as exc:
    self._reporter.increment(REJECTED_METRIC, tags={"view": view})   #
presentation.render_intent.rejected
    return AcceptedIntent(view=default, rejection=exc.detail)
return AcceptedIntent(view=view, options=dict(options))
```

Three rulings are visible in those five lines (ADR-K D-12):

1. **The intent is validated before anything is rendered** — an unknown view, an undeclared option or an out-of-range value never reaches a projection.
2. **A rejection is counted, not swallowed.** `presentation.render_intent.rejected` carries the offending view as a tag, so a model that keeps guessing the same wrong id is visible in telemetry rather than in a bug report.
3. **A rejection never ends the turn.** The endpoint's documented default is substituted and the conversation still shows something. A bad guess is a normal event, not a failure.

The `accept` / `render` split exists for a reason worth keeping: `accept` needs no portfolio, so validating a turn's surfaces costs no database read, and the price of a bad guess is not paid by every good one.

This seam is newer than the intent itself. Until GGU-84 the only thing a tool-emitted view id met was a regex on the string — which `totally.invented.v9` satisfies — and options were never looked at. `TurnRenderResolver` now stands between the tool and the client, and delegates the judging so the model's path and the client's path cannot drift apart.



---

5. What the model may and may not put in a payload

The render intent is the *only* thing the model contributes to a surface. It picks an id and fills declared options. It never defines a shape, never adds a field, and never supplies a figure — every number in the narration is a `*Display` string a deterministic tool returned and the model copied.

Captured from a real turn:

```
data: {"kind":"render-intent","index":0,
       "renderIntent":{"view":"attribution.driver.v1","options":{"symbol":"NVDA"}}}
data: {"kind":"message","parts":[{"kind":"text","text":"NVIDIA Corp (NVDA) contributed +$4,370.40
today: its price moved +4.20% (+$48.56) on 90 shares at $1,204.66, 134.88% of the day's
+$3,240.18."}], …}
```

`+$4,370.40`, `+4.20%`, `+$48.56`, `$1,204.66`, `134.88%`, `+$3,240.18` — all six are fields of the `attribution.driver.v1` payload the client then fetched. The model chose *what to show*. Arithmetic belongs to code that has tests.

---

6. `guidance` is a declaration too, and `null` is a real answer

Each catalog entry carries `guidance` — the `use_when` / `avoid_when` pair that teaches a model when to pick this view, and the generated `prompt` fragment that carries it into the system prompt.

```
$ … | python3 -c '…print(v["id"], v["guidance"] is None)'
order.ticket.v1          guidance: null
order.ticket.v2          guidance: {"use_when": "the user expresses buy or sell INTENT that resolves
to a
                                   concrete, editable order — including vague intent such as \"put
~$10k
                                   into semis, keep it diversified\"…", "avoid_when": "…"}
attribution.waterfall.v1 guidance: {"use_when": "the user asks WHY a value changed and the answer
                                   decomposes into signed contributions that sum to a net delta…"}
```

`null` is not an oversight to be filled in. Guidance is written by design, in the component's own page; where no page exists the contract prints that the page does not exist. A sentence invented by an engineer here would be indistinguishable, to every later reader, from one a designer approved — so the rule is to leave it unset (`.claude/rules/backend/presentation.md` §6).

---

7. A surface sends back a choice, never a sentence

The second way a payload varies is an **action**: what a rendered surface may send back to start a new turn (spec 0022). A projection declares its actions, and the catalog publishes them with their bounds:

```
"actions": [{
  "action":  "drill.attribution.v1",
  "label":   "Break this down",
  "arg":     "driver",
  "control": "choice",
  "choices": ["AAPL", "AGG", "AMD", "AVGO", "BND", "GLD", "JNJ", "MSFT",
              "NVDA", "PG", "PYPL", "TIP", "TSLA", "USD", "VNQ", "VOO", "XOM"],
  "pattern": null
}]
```

Two actions are registered today, and they are two shapes of the same rule:

Action	Argument	What the turn receives
`ask.suggested.v1`	`suggestion`, a registered key (`biggest-drivers`, `tax-lots`, `allocation`, `why-did-i-move`)	that key's **registered** prompt text, byte for byte, from `app.actions.suggestions`
`drill.attribution.v1`	`driver`, a symbol from the curated market reference	a sentence that is a **constant in the backend**, with the validated symbol substituted in

The client cannot send prompt text through either path. It sends an action id it did not invent plus a scalar drawn from a bound the payload published; the backend composes the sentence. A chip is a user turn nobody typed, so it cannot be an option — there is no view to re-render — and a drill is not the same chart with a parameter moved, it is the agent recomposing at a new depth. That is why both are turns and neither is an option.

Run, and refused:

```
$ … '{"kind":"data","data":{"action":"ask.suggested.v1","args":{"suggestion":"biggest-drivers"}}}'
data: {"kind":"render-intent","index":0,"renderIntent":{"view":"attribution.waterfall.v1","options":
{}}}
data: {"kind":"message","parts":[{"kind":"text","text":"Your portfolio is down -$8,248.86 (-1.68%)
over the last week…"}], …}

$ … '{"action":"drill.attribution.v1","args":{"driver":"FAKE"}}'
{"jsonrpc":"2.0","id":"3","error":{"code":-32602,
 "message":"action 'drill.attribution.v1' rejected argument(s) ['driver']"}}
```

Unlike a refused render intent, a refused action has **no fallback**: the stream never opens. A render intent is the model guessing, which must be survivable; an action is a client sending something it was never given, which must not be papered over.

The bound in `choices` is the same `Symbol` type `OrderTicketOptions.symbol` uses — two declarations of one fact, so a test compares them rather than a comment claiming they agree.

---

8. The component name is declared four times, on purpose

`component` is the single string the Flutter client registers a widget under. It is written down in four independent places:

1. the **design catalog** — a component's own A2UI schema block;
2. the projection's **A2UI adapter** in `backend/src/app/presentation/`;
3. the generated **JSON snapshot**, `docs/catalog/view-catalog.snapshot.json`;
4. the generated **Markdown contract**, `docs/catalog/CONTRACT.md`.

Until `backend/tests/unit/presentation/test_component_names.py` existed, nothing compared them — and spec 0021 §1 recorded the consequence: the backend emitted `WaterfallChart` and `DonutChart` while

design drew the same two under other names. **Both halves were internally consistent, so both gates stayed green.** Four of six names were wrong.

The test holds design's half as a deliberate literal:

```
DESIGN_CATALOG: Final[dict[str, str]] = {
    "attribution.waterfall.v1": "AttributionWaterfall",  # badge says WaterfallChart; Zone 2 wins
    "holdings.allocation.v1":   "HoldingsSummary",       # index card says AllocationSummary
    "order.ticket.v1":          "OrderTicket",
    "order.ticket.v2":          "MultiLegOrderTicket",   # v1 holds the name (0031 OD-1)
    "payoff.curve.v1":          "RiskPayoffChart",       # index card says PayoffChart
    …
}
```

That literal is the **one** inventory in the suite not read from the registry, because the whole point is to compare the backend against something outside it. A second transcription of it is forbidden. Adding a chart makes this file red until you add its name, on purpose (spec 0023): *a component nobody has named is a component the client cannot register.*

This is the general lesson, and it applies beyond names:

> **A drift check cannot tell you the generator is wrong.** Re-rendering proves an artefact is *current*; only a second, independent declaration proves it is *right*.

---

9. The generated artefacts, and why regeneration is the gate

Registering a projection once publishes it to five consumers: OpenAPI, the `GET /v1/views` catalog, the generated prompt fragments, the A2UI adapter, and the **frozen design contract**. The last is published in two renderings from **one** render pass (`presentation/catalog_artifacts.py`):

Artefact	For	Carries
`docs/catalog/CONTRACT.md`	the designer	options, field inventory, guidance, example, digest
`docs/catalog/view-catalog.snapshot.json`	the Flutter catalog test	component, view id, field paths, options schema, example, digest

```
cd backend && make catalog        # re-render both and write them
cd backend && make gate           # …which runs catalog-check: re-render, diff, fail on any difference
```

`catalog-check` sits inside `make gate` , between `budget` and `quality` , and it re-renders **every** artefact into a buffer and diffs each against the file on disk. A stale file fails the build and prints the command that fixes it. So a projection cannot change without the artefacts changing in the same commit, and a reviewer sees the moved field as a diff rather than discovering it after the design work is done.

A third rendering is **one entry in** `ARTIFACTS` **plus its renderer** — never a second script and never a second gate step. That is not tidiness: two artefacts gated by two scripts is how the two start disagreeing.

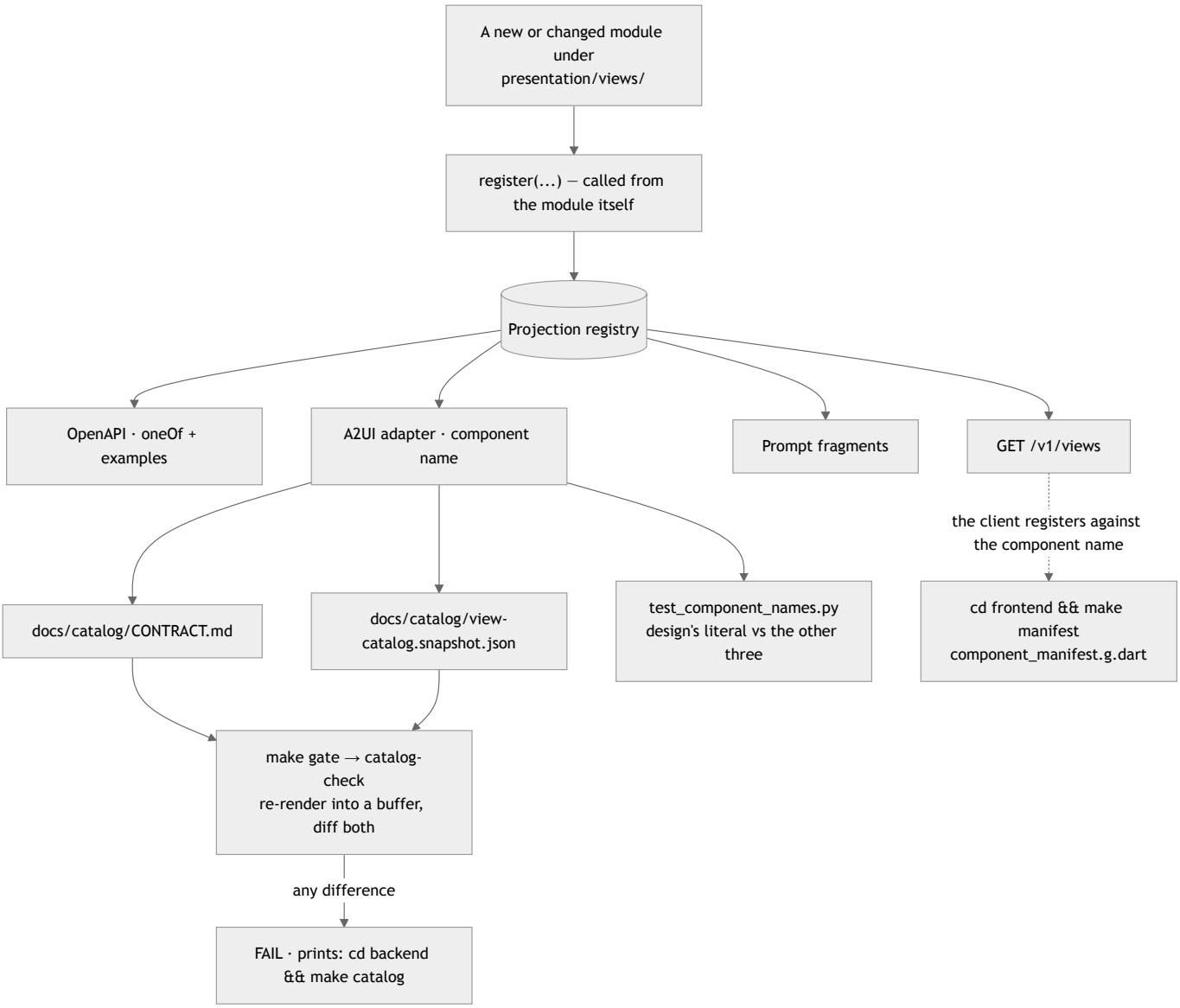
The digest

Each view carries a 12-character digest — the first 12 hex characters of a sha256 over its **id, summary, endpoints, options schema, payload schema and example**:

```
activity.timeline.v1      ActivityTimeline       dcb1032a4438
attribution.driver.v1     DriverDetail           13c7eae1516d
attribution.waterfall.v1  AttributionWaterfall   1f290bfc4970
holdings.allocation.v1    HoldingsSummary        fa2cf78e5248
holdings.positions.v1     PositionList           279142f5554a
order.ticket.v1           OrderTicket            d0daddcb8960
order.ticket.v2           MultiLegOrderTicket    e6c713d6a144
payoff.curve.v1           RiskPayoffChart        7066577b5390
portfolio.summary.v1      PortfolioSummary       a7389adf90d7
scenario.comparison.v1    ScenarioComparison     6f73ed9fad79
taxlots.table.v1          TaxLotTable            c960b55bd552
```

If a `v1` digest moves, something that was frozen changed. It is the version identifier the prompt fragments and the Figma UI Kit cite, so a designer and a model can both say *which* `v1` they built against.

Two keys are deliberately **excluded** from the hash: `fields` and `prompt`. Both are *renderings* of facts the digest already covers — the field inventory is the payload schema walked, the prompt fragment is the summary, options and guidance joined. Hashing them too would move every frozen digest the day a rendering changed its wording, while nothing a client binds had moved.



The client's half

`frontend/` runs the same idea over its own generated file. One directory per component under `app/lib/components/`, and the wiring is generated:

```
cd frontend && make manifest        # rewrite app/lib/components/component_manifest.g.dart
cd frontend && make gate            # …which runs manifest-check and fails on a stale one
```

```
// GENERATED FILE — do not edit by hand.
void registerCatalogComponents(ComponentRegistry registry) {
  attribution_driver.register(registry);
  attribution_waterfall.register(registry);
  …
}
```

Adding a component edits no existing file (ADR-L D-3) — the same rule as the backend's registry, enforced the same way: by regeneration, never by trust. Eight of the eleven views have a component in this build; a

view with none is a placeholder the client skips, not an error.

---

10. How a client tells one kind of field from another

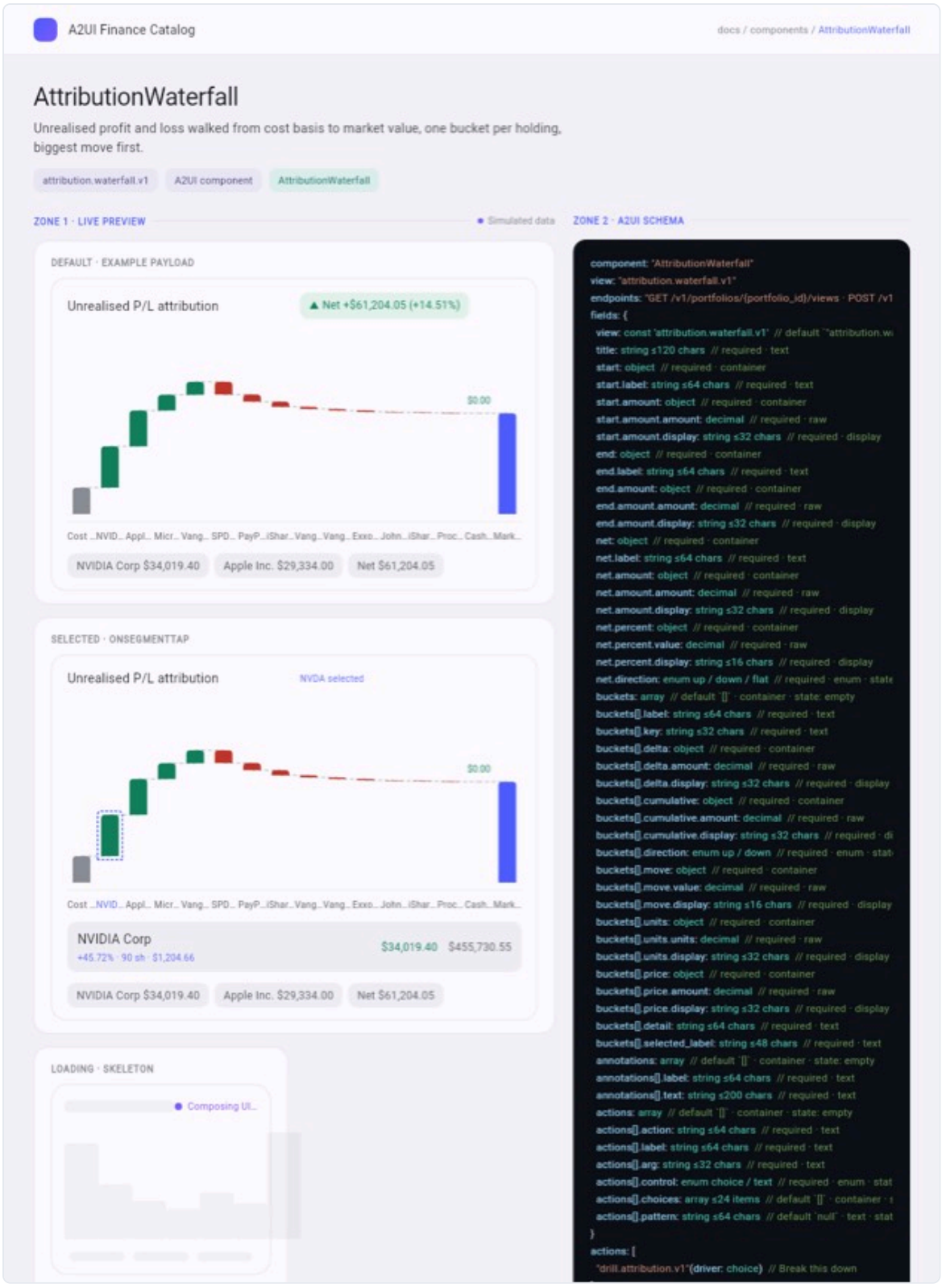
A client that wants to be generic — a catalog gallery, a test harness, a documentation tool — reads `fields[]` from `GET /v1/views` and never guesses from a field's name. Three columns answer three questions.

`binding` **— what is this value for?** Derived from structure, never from a hardcoded name:

Value	The payload member	The client
`display`	the formatted half of an ADR-E pair	prints it, verbatim
`raw`	the exact decimal half (`amount` / `value` / `units`)	uses it for geometry only — never prints it (N-4)
`text`	a server-written label, caption, detail or note	prints it, verbatim; never drops it (N-9)
`enum`	a bounded token (`up` / `down` / `flat` , `buy` / `sell` , `none` / `over_cash`)	selects a variant: colour, icon, tone
`constant`	the `view` discriminator	matches on it; never renders it
`container`	an object or array holding other fields	walks into it

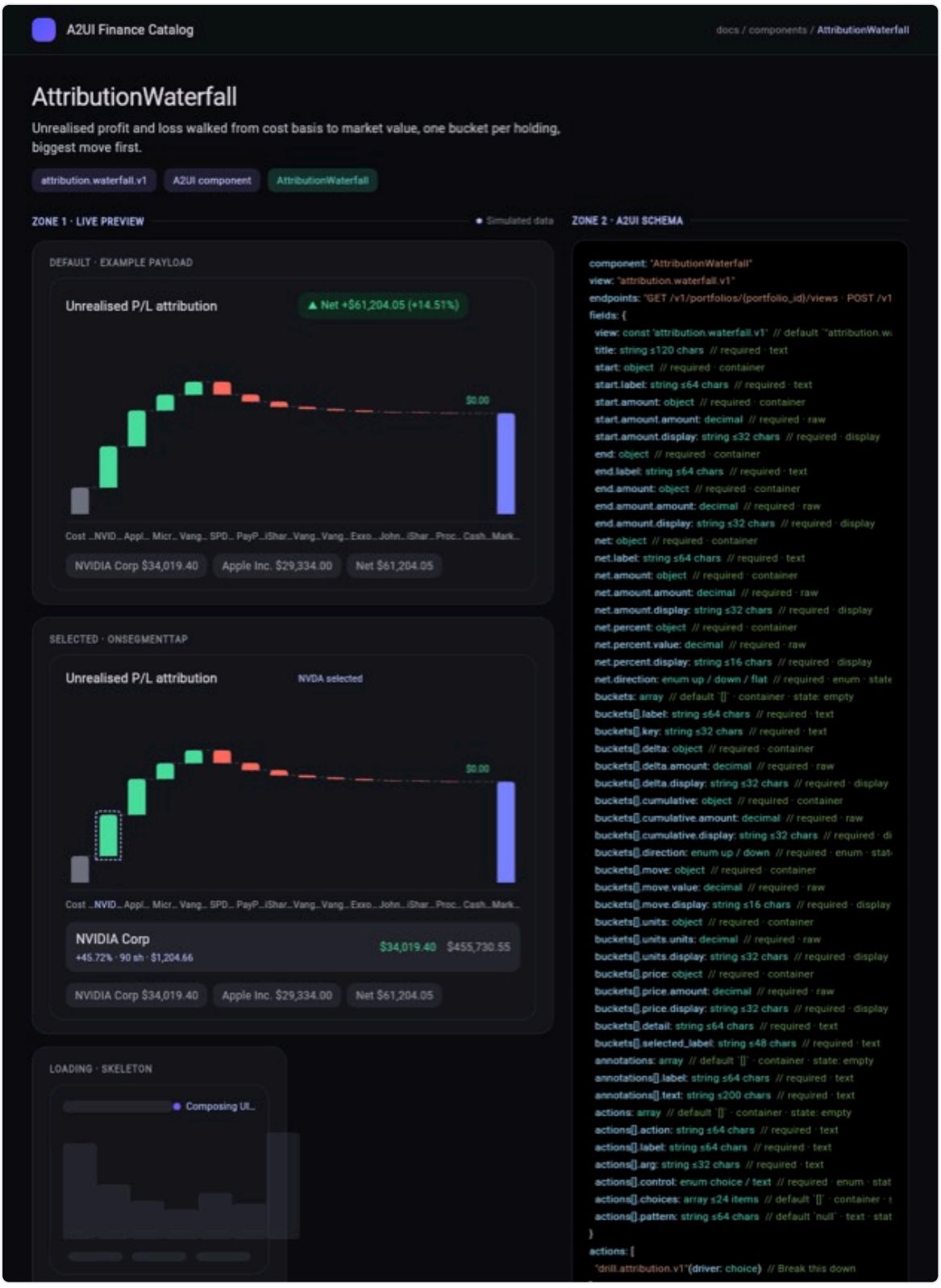
Pre-formatted is derived from the pair, not from the name `display` — keep shipping a string beside a figure and a new view classifies itself.

`state` **— which UI state can this field drive?** An array can be `empty` , a nullable field can be `null` , an enum or boolean selects a `variant` . That is how the gallery knows a component needs an empty state, a loading skeleton and a variant row without anyone listing them:



The catalog page for AttributionWaterfall: live preview, selected state and loading skeleton beside the A2UI schema block

The theme is the client's own; the payload is identical in both. Colour comes from the `enum` rows (`direction`, `trend`, `tone`), never from parsing a `display` string, which is what lets one payload serve two themes:



The same AttributionWaterfall page in the dark theme, rendered from the same payload

`input` **— may a user edit this?** Non-null on exactly the fields named after an option, carrying the bound to render the control from. Use that bound; do not add a stricter client rule beside it (N-8).

Read together, those three columns are enough to render a component you have never seen: print the `display` and `text` rows, plot the `raw` rows, switch on the `enum` rows, offer a control for the `input` rows, and handle the `state` each row declares.

11. Adding a chart — the sequence

The authoritative checklist is `.claude/rules/backend/presentation.md` §6. In the order the work actually happens:

1. **Write the spec first.** `specs/backend/NNNN-….md`, approved before code.
2. **One new module** under `presentation/views/`, id versioned `v1`. It may grow into a small group of files beside `views/` — the 150-line budget is a gate, "one module" is a preference — but the package requires exactly one `register(...)` per module in `views/`.
3. **Options model**: bounded, defaulted, described, with examples. Reuse the mixins in `options.py`.
4. **DTO** built from `fragments.py`, with `view: Literal["<id>"]` and every number as an ADR-E pair.
5. `render()` **pure** — no I/O, no repository, no session, no clock read — and unit-tested through every option branch and the empty/edge data cases.
6. **A2UI adapter + its test.** The component name comes from the design catalog, and nowhere else.
7. `register(...)` **from the module itself.** Nothing else is edited. If you find yourself adding to an enum, a union list or an `if/elif`, stop: the seam is wrong (ADR-K D-15).
8. **Add the name to** `DESIGN_CATALOG` in `test_component_names.py`. That file is red until you do.
9. **Guidance** in `<view>_guidance.py` *if design has written a page*. Otherwise leave it unset.
10. `cd backend && make catalog` and commit both artefacts.
11. `cd frontend && make manifest` if the client ships a widget for it.
12. `cd backend && make gate` and `cd frontend && make gate` — green, or it is not done.

What you should *not* have edited by the end: `registry.py`, any controller, any union list, `CONTRACT.md` or the snapshot by hand.

---

12. Where to go next

- **Feeding charts** — the endpoints, the envelope, the ETag, the agent stream, and every projection as served.
- `docs/catalog/CONTRACT.md` — the frozen field inventory with digests.
- `docs/diagrams/06-response-shaping.md` — who chooses the shape · `05-agent-turn.md` — the turn in full · `12-frontend-layers.md` — what the client may import.
- ADR-K — the reasoning, including the projection DSL we declined and why.